

RAPPORT DE LA VUE SMA DU PROJET PARALLAX

RÉSUMÉ

Ce rapport présente la conception et la modélisation du système PARALLAX, une infrastructure de calcul distribué développée dans le contexte académique de l'École Nationale Supérieure Polytechnique de Yaoundé. Face aux contraintes d'accès aux ressources cloud distantes — notamment les coûts élevés, l'instabilité du réseau et la dépendance à Internet — PARALLAX exploite un ensemble de machines locales hétérogènes et sous-utilisées afin de fournir une capacité de calcul souveraine et à faible coût.

L'approche retenue repose sur le paradigme des Systèmes Multi-Agents (SMA), qui permet de représenter chaque composant du cluster comme un agent autonome doté d'un rôle, d'un comportement et de mécanismes de communication clairement définis. Trois agents principaux structurent le système : l'Agent Master, chargé de la décomposition et de la distribution des calculs ; l'Agent Worker, responsable de l'exécution locale des sous-tâches ; et l'Agent Contrôleur, garant de la supervision et de la résilience du cluster.

La modélisation s'appuie sur l'architecture BDI (*Belief-Desire-Intention*) pour les agents cognitifs, complétée par une approche réactive à état pour l'Agent Worker.

Les travaux présentés couvrent l'analyse des exigences, l'identification des agents et de leurs rôles, la définition des protocoles de communication et de coordination, ainsi que les mécanismes de tolérance aux pannes. L'ensemble constitue une base formelle solide pour le développement et l'évolution de cette infrastructure académique de calcul distribué.

ABSTRACT

This report presents the design and modelling of the PARALLAX system, a distributed computing infrastructure developed in the academic context of the National Advanced School of Engineering of Yaoundé. Faced with the constraints of accessing remote cloud resources — high costs, network instability and Internet dependency — PARALLAX leverages a pool of heterogeneous, under-utilised local machines to provide sovereign and low-cost computing power.

The chosen approach is based on the Multi-Agent Systems (MAS) paradigm, where each cluster component is modelled as an autonomous agent with a defined role, behaviour, and communication mechanisms. Three main agents structure the system : the Master Agent, responsible for task decomposition and distribution ; the Worker Agent, handling local execution of sub-tasks ; and the Controller Agent, ensuring cluster supervision and resilience.

The modelling relies on the BDI (*Belief-Desire-Intention*) architecture for cognitive agents, complemented by a state-based reactive approach for the Worker Agent.

The work presented covers requirements analysis, agent and role identification, definition of communication and coordination protocols, and fault-tolerance mechanisms. Together, these elements provide a solid formal foundation for the development and evolution of this academic distributed computing infrastructure.

TABLE DES MATIÈRES

Résumé	1
Abstract	2
Sigles et Abréviations	6
Glossaire	7
Liste des Figures	8
Liste des Tableaux	10
1 Généralités et État de l’art	12
1.1 Généralités	12
1.1.1 Concepts de base	12
1.2 État de l’art	14
1.2.1 Architectures centralisées	14
1.2.2 Architectures hiérarchiques	14
1.2.3 Approches orientées systèmes multi-agents	15
1.2.4 Bilan comparatif	15
1.2.5 Positionnement	15
2 Analyse et conception	17
2.1 Analyse	17
2.1.1 Fonctionnalités et exigences	17
2.1.2 Diagramme de cas d’utilisation	18
2.1.3 Identification des agents	23
2.1.4 Identification des rôles	24
2.1.5 Identification du SMA	24
2.2 Conception	30

2.2.1	Architecture du SMA	31
2.2.2	Structure de l'agent et du SMA	31
2.2.3	Comportement des agents	36
2.2.4	Description des rôles	39
2.2.5	Protocoles de communication et coordination	47
3	Implémentation et Résultats	51
3.1	Conception et limites du modèle réseau classique (Network Agent Abstraction)	51
3.2	L'Agent Réceptionniste	52
3.2.1	Architecture multi-thread de l'Agent Réceptionniste	52
3.2.2	Cycle de vie et découverte de l'Agent Réceptionniste	53
3.3	L'Agent Master	53
3.3.1	Architecture multi-thread de l'Agent Master	54
3.3.2	Workflow complet d'Orchestration et d'Exécution	54
3.4	L'Agent Worker	56
3.4.1	Architecture multi-thread de l'Agent Worker	57
3.4.2	Compilation Dynamique, Isolation et Rotation des Logs	57
3.5	L'Agent Contrôleur	58
3.5.1	Architecture multi-thread de l'Agent Contrôleur	58
3.5.2	Mécanisme de Watchdog et Détection de pannes	59
3.6	Trace complète d'un cycle de calcul	59
	Conclusion Générale	62
	Références	62

TABLE DES MATIÈRES

SIGLES ET ABRÉVIATIONS

Sigle	Signification
API	Application Programming Interface
AST	Abstract Syntax Tree
CLI	Command Line Interface
CPU	Central Processing Unit
DAG	Directed Acyclic Graph
DDR	Double Data Rate
DHCP	Dynamic Host Configuration Protocol
FIFO	First In, First Out
GB	Gigabyte
GHz	Gigahertz
IP	Internet Protocol
LAN	Local Area Network
LLM	Large Language Model
MAC	Media Access Control
MB	Megabyte
MPI	Message Passing Interface
RAM	Random Access Memory
RJ45	Registered Jack 45
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
UUID	Universally Unique Identifier
ACL	Agent Communication Language
BDI	Belief-Desire-Intention
HB	Heartbeat
IaaS	Infrastructure as a Service
IO	Input/Output
OS	Operating System
SGBD	Système de Gestion de Base de Données
RPC	Remote Procedure Call
SMA	Système Multi-Agent

GLOSSAIRE

- * **Annotation** : Directive spéciale insérée dans le code source par le développeur pour indiquer au système comment décomposer et distribuer une tâche (ex : `@parallax.split`).
- * **Agent cognitif** : Agent capable de raisonnement symbolique basé sur des croyances, des objectifs et des intentions (BDI).
- * **Agent réactif** : Agent dont le comportement repose uniquement sur des règles condition-action sans raisonnement explicite.
- * **Architecture agentique** : Organisation d'un système logiciel sous forme d'agents autonomes interagissant dans un environnement partagé.
- * **Cluster** : Ensemble de machines interconnectées coopérant pour exécuter des tâches distribuées.
- * **Contrôleur** : Nœud du cluster chargé de la supervision globale. Il reçoit les messages de heartbeat, vérifie l'état des machines en temps réel et détecte les pannes.
- * **Décomposition de tâche** : Processus consistant à transformer une tâche globale en sous-tâches exécutables en parallèle.
- * **Élection distribuée** : Mécanisme permettant de sélectionner dynamiquement un nœud leader sans contrôle central unique.
- * **Futex** : Mécanisme de synchronisation bas niveau permettant de bloquer un thread jusqu'à ce qu'une condition soit satisfaite.
- * **Gossip** : Méthode de communication où chaque nœud échange périodiquement des informations avec un sous-ensemble aléatoire du cluster.
- * **Heartbeat** : Signal périodique émis par les nœuds pour indiquer leur état et leur présence.
- * **Horloge logique de Lamport** : Mécanisme permettant d'ordonner les événements dans un système distribué sans horloge globale fiable.
- * **Latence réseau** : Temps nécessaire pour transmettre un message entre deux nœuds dans un réseau distribué.
- * **Nœud** : Machine physique ou virtuelle appartenant au cluster, exécutant des agents et des tâches distribuées.
- * **Nœud maître (Master)** : Nœud responsable de la coordination, de la distribution des tâches et de l'agrégation des résultats.
- * **Nœud worker** : Nœud exécutant les sous-tâches attribuées par le maître.
- * **Orchestration** : Processus de coordination de l'exécution des tâches dans un système distribué.
- * **Quorum** : Nombre minimal de nœuds requis pour valider une opération distribuée.

- * **Score pondéré** : Fonction combinant plusieurs métriques (CPU, RAM, latence, etc.) pour évaluer un nœud.
- * **Split-brain** : Situation où plusieurs nœuds deviennent leaders simultanément à cause d'une partition réseau.
- * **Système Multi-Agent (SMA)** : Paradigme où des agents autonomes interagissent dans un environnement distribué.
- * **Tolérance aux pannes** : Capacité d'un système à continuer de fonctionner malgré des défaillances.
- * **Thread** : Unité d'exécution légère au sein d'un processus, partageant la même mémoire que les autres threads du processus.
- * **Vue globale du système** : Représentation agrégée de l'état du cluster à un instant donné.

TABLE DES FIGURES

2.1	Diagramme de Contexte	17
2.2	Diagramme de cas d'utilisation	18
2.3	Diagramme de séquences système — Importer une tâche	26
2.4	Diagramme de séquences système — Soumettre une tâche	27
2.5	Diagramme de séquences système — Voir les résultats	28
2.6	Diagramme de séquences système — Authentification	29
2.7	Diagramme de séquences système — Consulter une machine	30
2.8	Architecture agentique de SMA_PARALLAX	31
2.9	Diagramme de classes	34
2.10	Diagramme d'état d'un nœud	37
2.11	Diagramme d'état d'une tâche	38
2.12	Diagramme d'état d'un programme	39
2.13	Interaction entre le rôle de réception et son environnement	40
2.14	Interaction entre le réceptionniste et le générateur de sous-tâches	40
2.15	Interaction détaillée entre réception et génération des sous-calculs	40
2.16	Interaction entre le générateur de sous-calculs et l'orchestrateur	41
2.17	Interaction entre les agents d'exécution et le synthétiseur	42
2.18	Interaction entre le synthétiseur et le réceptionniste	43
2.19	Diagramme de séquences technique — Authentification	43
2.20	Diagramme de séquences technique — Création de compte	44
2.21	Diagramme de séquences technique — Importation d'une tâche	44
2.22	Diagramme de séquences technique — Exécution d'une tâche	45
2.23	Diagramme de séquences technique — Consultation des machines	45
2.24	Diagramme de séquences technique — Consultation des tâches	46
2.25	Diagramme de séquences technique — Suppression d'un nœud	46
2.26	Architecture à deux couches du heartbeat	47
2.27	Machine à états — suivi d'un pair par un observateur	49

2.28	Diagramme des composants	50
3.1	Structure des threads de l'Agent Réceptionniste	53
3.2	Structure des threads de l'Agent Master	54
3.3	Diagramme d'activité : Workflow d'exécution de l'Agent Master	56
3.4	Structure des threads de l'Agent Worker	57
3.5	Structure des threads de l'Agent Contrôleur	59
3.6	Diagramme d'activité : Workflow d'exécution de l'Agent Master	61

LISTE DES TABLEAUX

1.1	Comparaison des approches de calcul distribué	15
2.1	Cas d'utilisation : Authentification	19
2.2	Cas d'utilisation : Création de compte	19
2.3	Cas d'utilisation : Importation d'une tâche	20
2.4	Cas d'utilisation : Soumission d'une tâche	20
2.5	Cas d'utilisation : Consultation des résultats	21
2.6	Cas d'utilisation : Consultation d'une machine	21
2.7	Cas d'utilisation : Suppression d'une machine du cluster	22
2.8	Cas d'utilisation : Consultation d'un programme en cours d'exécution . . .	23
2.9	Identification des agents	23
2.10	Identification des rôles	24
2.11	Typologie des agents de SMA_PARALLAX	33
2.12	Table des actions de SMA_PARALLAX	35
2.13	Table des événements de SMA_PARALLAX	36
2.14	Types de messages dans SMA_PARALLAX	47

1. GÉNÉRALITÉS ET ÉTAT DE L'ART

1.1 Généralités

Cette section présente les notions fondamentales nécessaires à la compréhension de ce travail, notamment celles liées au calcul distribué, au parallélisme et aux systèmes multi-agents. Ces concepts constituent les bases théoriques sur lesquelles repose le système développé.

1.1.1 Concepts de base

- **Calcul distribué** : Le calcul distribué désigne une approche informatique consistant à répartir l'exécution d'un calcul sur plusieurs machines interconnectées. L'objectif principal est d'améliorer les performances, la scalabilité et la tolérance aux pannes en exploitant la puissance de calcul collective d'un ensemble de nœuds [3].
- **Parallélisme** : Le parallélisme consiste à exécuter simultanément plusieurs opérations ou tâches afin de réduire le temps global d'exécution. Il peut être basé sur les données ou sur les instructions, selon la structure du problème à résoudre [3].
- **Agent** : Selon Wooldridge et Jennings, un agent est un système informatique situé dans un environnement et qui agit d'une façon autonome pour atteindre les objectifs (buts) pour lesquels il a été conçu [4]. Il s'agit d'une entité logicielle autonome capable de percevoir son environnement, de prendre des décisions et d'agir en vue d'atteindre des objectifs définis.
- **Caractéristiques d'un agent** : Un agent possède généralement les propriétés suivantes [4, 8] : il est *situé* (capable d'agir sur son environnement à partir des entrées sensorielles qu'il reçoit), *autonome* (capable d'agir sans intervention d'un tiers et contrôlant ses propres actions), *proactif* (exhibant un comportement opportuniste et prenant l'initiative au bon moment), *capable de répondre à temps* (percevant son environnement et élaborant une réponse dans les délais requis), et *social* (capable d'interagir avec d'autres agents afin d'accomplir des tâches).
- **Typologie des agents** : Le cours de Génie Informatique de l'ENSPY [9] distingue quatre types principaux d'agents. Les *agents réactifs simples* répondent directement aux stimuli de l'environnement, sans mémoire interne, par règles condition-action. Les *agents réactifs avec état* conservent une représentation partielle de l'environnement afin d'adapter leurs actions au contexte. Les *agents cognitifs orientés but* prennent leurs décisions en fonction d'objectifs à atteindre. Les *agents cognitifs orientés utilité* choisissent les actions maximisant un critère d'utilité ou de performance [4].

- **Système multi-agents** : Un système multi-agents est un système distribué composé d'un ensemble d'agents [8]. Chaque agent possède des informations ou des capacités de résolution de problèmes limités (point de vue partiel) ; il n'y a aucun contrôle global du système ; les données sont décentralisées ; et le calcul est asynchrone [8]. Les agents coopèrent à travers des mécanismes de communication et de coordination pour résoudre un problème global.
- **Communication entre agents** : Dans un système multi-agents, les agents échangent des informations à travers des protocoles de communication permettant la coordination, la synchronisation et le partage d'état [4, 8]. Ces interactions se déclinent en coordination, communication et coopération.
- **Architecture BDI** : L'architecture BDI (*Belief-Desire-Intention*), fondée sur la théorie de l'action rationnelle de Michael Bratman [9], structure l'état mental d'un agent en trois composantes : les *croiances* (informations que l'agent possède sur l'environnement, pouvant être incorrectes ou incomplètes), les *désirs* (états que l'agent aimerait voir réalisés) et les *intentions* (désirs que l'agent a décidé d'accomplir) [9, 4].
- **Coordination** : La coordination désigne l'ensemble des mécanismes permettant aux agents d'organiser leurs actions afin d'éviter les conflits, répartir les tâches et atteindre un objectif collectif [4, 8].
- **Environnement** : L'environnement représente l'espace dans lequel évoluent les agents. Il correspond à l'ensemble des ressources, événements et informations accessibles aux agents du système [4].
- **Orchestrateur** : L'orchestrateur est un composant chargé de la répartition et de la coordination des tâches entre les différents agents ou nœuds du système. Il optimise l'utilisation des ressources disponibles et assure une exécution efficace des calculs [3, 6].
- **Contrôleur** : Le contrôleur est un nœud spécialisé chargé de la supervision continue du cluster. Il centralise les informations envoyées périodiquement par les autres nœuds et maintient une vue globale de l'état du système afin de détecter les anomalies ou défaillances [3].
- **Ordonnanceur** : L'ordonnanceur est responsable de la planification de l'exécution des tâches. Il décide de l'affectation des ressources en fonction de critères tels que la charge, la disponibilité ou la priorité [5].
- **Tâche** : Une tâche représente une unité de travail à exécuter dans le système. Dans un contexte distribué, une tâche peut être découpée en sous-tâches afin d'être exécutée en parallèle sur plusieurs nœuds [5].
- **DAG (Directed Acyclic Graph)** : Un DAG est un graphe orienté sans cycle représentant les dépendances entre tâches. Il permet de modéliser l'ordre d'exécution des calculs dans les systèmes parallèles et distribués [5].
- **Heartbeat** : Le heartbeat est un mécanisme de communication périodique permettant à un nœud ou un agent d'indiquer qu'il est actif. Il est utilisé pour la supervision et la détection de pannes dans les systèmes distribués [3, 6].
- **Remplaçant** : Le remplaçant est un nœud désigné pour reprendre automatiquement le rôle d'un composant critique en cas de défaillance afin d'assurer la continuité du système [7].

- **Broadcast** : Le broadcast est un mécanisme de communication consistant à diffuser un message à l'ensemble des nœuds ou agents du système [3].

1.2 État de l'art

L'accès à des ressources de calcul à faible coût dans des environnements à connectivité limitée constitue un problème récurrent dans les contextes académiques et scientifiques. Une approche couramment adoptée consiste à mutualiser des ressources matérielles existantes afin de construire une infrastructure de calcul distribuée. Cette mutualisation peut être réalisée selon différentes architectures de coordination des composants du système, principalement : les architectures centralisées, les architectures hiérarchiques et les approches orientées systèmes multi-agents.

1.2.1 Architectures centralisées

Les architectures centralisées reposent sur un point de coordination unique chargé de la gestion globale du système. Ce nœud central assure la réception des tâches, leur planification et leur distribution vers les ressources de calcul disponibles.

HTCondor [2] illustre ce modèle. Il s'agit d'un système de gestion de tâches à haute capacité permettant d'exploiter des cycles de calcul inactifs sur un ensemble de machines hétérogènes. Un ordonnanceur central gère les files de tâches et attribue dynamiquement les travaux aux machines disponibles en fonction de leur état et de leurs capacités.

Cette approche présente l'avantage d'une conception simple et d'une vision globale unifiée de l'état du système. Elle permet une prise de décision cohérente et centralisée.

Cependant, elle introduit plusieurs limitations : le nœud central constitue un point unique de défaillance, la montée en charge peut engendrer un goulot d'étranglement, et la variabilité des environnements hétérogènes complexifie la politique d'ordonnancement.

1.2.2 Architectures hiérarchiques

Les architectures hiérarchiques structurent le système en plusieurs niveaux de coordination. Un nœud de contrôle global supervise des entités intermédiaires, lesquelles gèrent à leur tour des ensembles de ressources locales.

Ce modèle est fréquemment utilisé dans les systèmes de grille de calcul et certaines infrastructures distribuées à grande échelle, comme BOINC [1] ou les grilles académiques, où la séparation en domaines de gestion permet de répartir la charge de coordination.

Cette organisation améliore la scalabilité par rapport aux architectures strictement centralisées et réduit la charge du niveau supérieur. Cependant, elle introduit une complexité supplémentaire liée à la coordination entre niveaux. La défaillance d'un nœud intermédiaire peut isoler une partie du système.

1.2.3 Approches orientées systèmes multi-agents

Les approches orientées systèmes multi-agents constituent un paradigme de modélisation des systèmes distribués basé sur la décomposition en entités autonomes appelées agents. Chaque agent possède un état local, un ensemble de comportements, ainsi que des capacités de perception et d'action sur son environnement [4, 8].

Dans le domaine du calcul distribué, cette approche permet de représenter explicitement les composants fonctionnels du système sous forme d'agents interagissant via des protocoles définis. La coordination globale résulte des interactions entre agents plutôt que d'une unique entité centrale. Elle est notamment utilisée dans des architectures inspirées du modèle InterRaP de Muller [9], qui combine comportement réactif et planification rationnelle à travers trois niveaux : comportement réactif et procédural, planification locale, et coopération entre agents.

Cette approche offre un cadre de modélisation plus explicite des interactions et des rôles du système. Elle facilite la représentation de systèmes hétérogènes et dynamiques, mais introduit une complexité de conception plus élevée.

1.2.4 Bilan comparatif

Critères	Centralisé	Hiérarchique	SMA
Tolérance aux pannes	✗	△	✓
Scalabilité	△	✓	✓
Gestion de l'hétérogénéité	△	△	✓
Faible dépendance réseau	✗	△	✓
Complexité de conception	✓	△	✗
Cohérence globale	✓	△	✓

TABLE 1.1 – Comparaison des approches de calcul distribué

Légende : ✓ satisfaisant △ partiellement satisfaisant ✗ insuffisant

Globalement, les architectures centralisées privilégient la simplicité et la cohérence au détriment de la robustesse. Les architectures hiérarchiques améliorent la scalabilité mais introduisent une dépendance forte à la structuration en niveaux. Les approches multi-agents offrent une meilleure flexibilité et une meilleure adaptation à l'hétérogénéité des ressources, au prix d'une complexité de conception plus importante.

1.2.5 Positionnement

Le projet PARALLAX s'inscrit dans une logique d'exploitation des ressources locales disponibles dans un environnement académique contraint.

Les architectures centralisées et hiérarchiques présentent des limites en termes de dépendance structurelle et de flexibilité dans des environnements fortement hétérogènes et soumis à des contraintes de connectivité.

L'approche orientée systèmes multi-agents est retenue comme cadre de modélisation du système, car elle permet de représenter les différents composants du cluster sous forme d'entités autonomes interagissant entre elles. Cette modélisation facilite la structuration des rôles (nœuds de calcul, coordination, supervision) et permet de définir des mécanismes de répartition et d'exécution des tâches reposant sur les interactions entre agents.

2. ANALYSE ET CONCEPTION

2.1 Analyse

2.1.1 Fonctionnalités et exigences

Le système PARALLAX implique deux catégories d'acteurs. Le **Chercheur** (étudiant, enseignant ou chercheur de l'École Nationale Supérieure Polytechnique de Yaoundé) soumet ses programmes annotés via une interface CLI et consulte les résultats d'exécution. Le **Gestionnaire de cluster** administre l'infrastructure via une interface Web : il supervise l'état des nœuds grâce aux heartbeats, et gère l'ajout ou le retrait de machines du cluster.

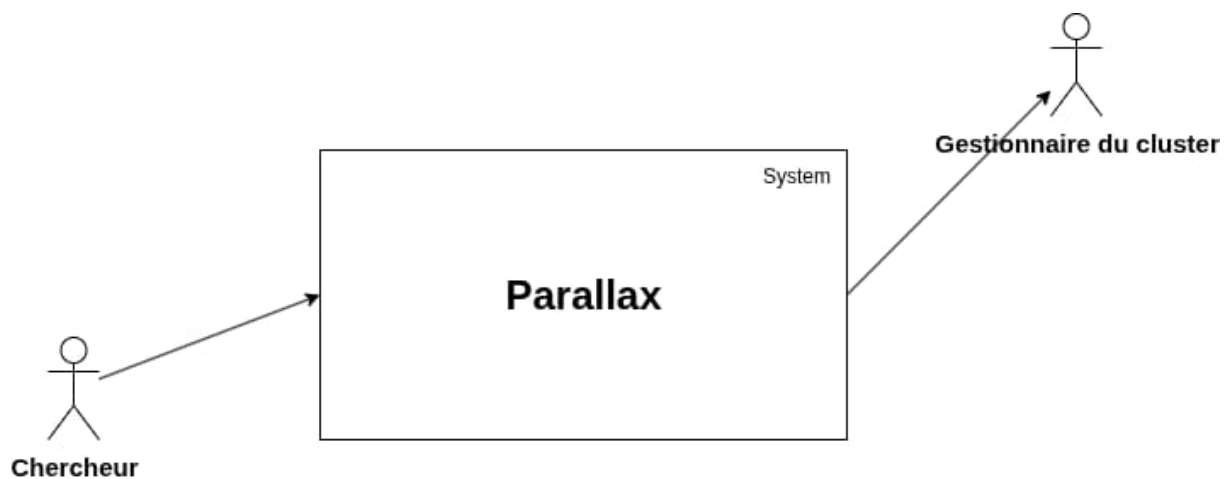


FIGURE 2.1 – Diagramme de Contexte

Exigences fonctionnelles :

- Soumission et exécution distribuée de tâches de calcul (Chercheur).
- Import et annotation de programmes via interface CLI (Chercheur).
- Consultation des résultats d'exécution avec détails (durée, date) (Chercheur).
- Supervision en temps réel de l'état des nœuds via interface Web (Gestionnaire de cluster).
- Gestion dynamique du cluster : ajout et retrait de machines (Gestionnaire de cluster).

Exigences non fonctionnelles :

- Résilience aux pannes : détection automatique et réaffectation des tâches, y compris en cas de surcharge d'un nœud.
- Transparence pour le développeur : parallélisation via annotations sans gestion manuelle du réseau.
- Légèreté : adaptation aux petits clusters hétérogènes à ressources modestes.
- Performance : optimisation de la distribution des tâches en fonction d'un score pondéré des nœuds (CPU, RAM, latence, fréquence), et prise en compte de la latence réseau afin de minimiser les temps de communication entre les machines.

2.1.2 Diagramme de cas d'utilisation

Le domaine du calcul distribué académique présente six exigences fondamentales. Le système doit fournir de la puissance de calcul pour les simulations et l'entraînement de modèles d'intelligence artificielle (ED-1). Il doit réduire les coûts d'infrastructure en proposant une alternative aux plateformes cloud distantes (ED-2). Il doit s'affranchir de la dépendance à une connexion Internet souvent instable dans le contexte camerounais (ED-3). Il doit permettre la mutualisation des machines existantes sous-utilisées sur le campus (ED-4). Il doit offrir une soumission de programmes sans expertise réseau, via un mécanisme d'annotations (ED-5). Enfin, il doit assurer la supervision et la maintenance continues du cluster par un administrateur (ED-6).

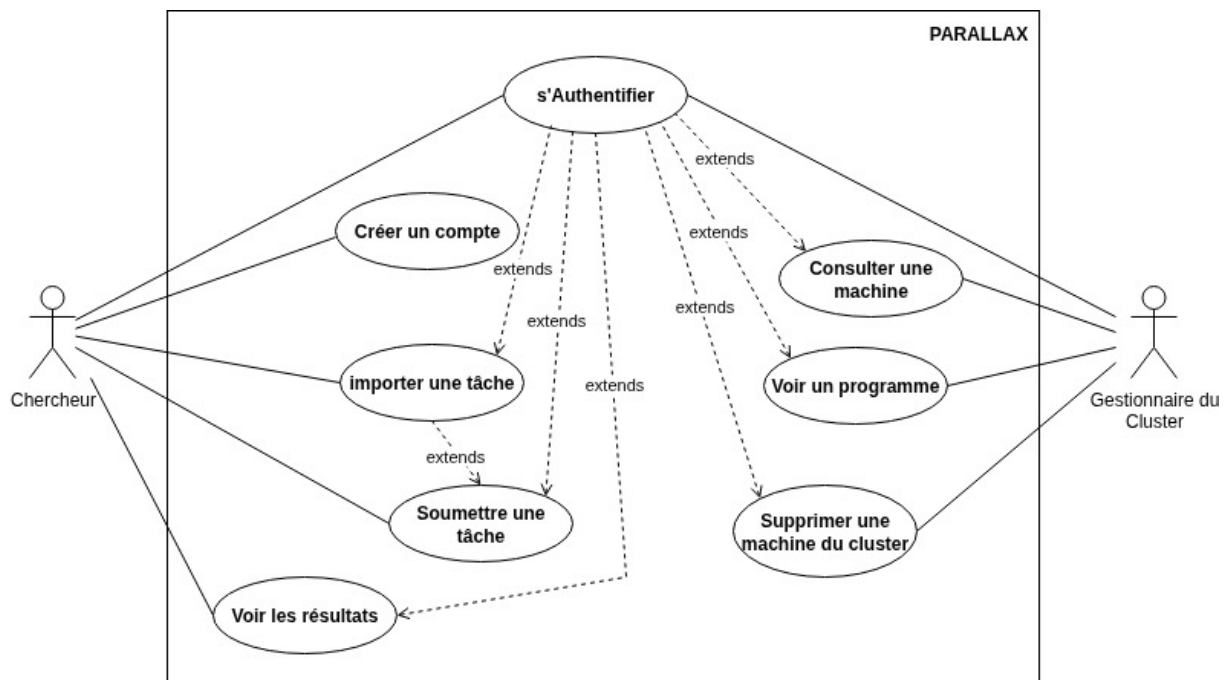


FIGURE 2.2 – Diagramme de cas d'utilisation

Le cas d'utilisation « S'authentifier » constitue le point d'entrée commun. Les cas « Créer un compte », « Importer une tâche », « Soumettre une tâche » et « Voir les résultats » concernent le Chercheur. Les cas « Consulter une machine », « Voir un programme » et « Supprimer une machine du cluster » concernent le Gestionnaire du cluster.

Objectif	Accéder aux fonctionnalités du système
Acteur(s)	Chercheur, Gestionnaire du cluster
Pré-condition	Aucune
Scénario nominal	1. L'acteur entre ses informations d'identification dans le système. 2. Le système vérifie l'exactitude des informations. 3. Les informations sont correctes, le système accorde l'accès aux fonctionnalités correspondantes au statut de l'utilisateur.
Scénarios alternatifs	— A1 : Erreur de saisie — Le système informe l'utilisateur et demande de nouvelles informations. — A2 : Erreur de connexion à la base de données — Le système renvoie un message d'erreur.
Post-condition	Accès aux fonctionnalités liées au statut de l'utilisateur

TABLE 2.1 – Cas d'utilisation : Authentification

Objectif	Créer un compte utilisateur
Acteur(s)	Chercheur
Pré-condition	Être authentifié
Scénario nominal	1. L'acteur entre ses informations dans le formulaire et le soumet au système. 2. Le système vérifie l'exactitude des informations. 3. Le système sauvegarde les informations et renvoie un message de succès.
Scénarios alternatifs	— A1 : Erreur de saisie — Le système demande à l'utilisateur de ressaisir ses informations. — A2 : Erreur de connexion à la base de données — Le système renvoie un message d'erreur.
Post-condition	Création du compte utilisateur

TABLE 2.2 – Cas d'utilisation : Création de compte

Objectif	Importer une tâche annotée
Acteur(s)	Chercheur
Pré-condition	Être authentifié
Scénario nominal	1. L'utilisateur clique sur l'option d'importation d'une tâche. 2. Le système lui permet de sélectionner le dossier ou fichier annoté via un sélecteur de ressources. 3. L'utilisateur sélectionne et valide. 4. La tâche est enregistrée dans le système.
Scénarios alternatifs	— A1 : Document invalide — Le système annule l'importation et renvoie un message d'erreur.
Post-condition	Dossier / fichier importé dans le système

TABLE 2.3 – Cas d'utilisation : Importation d'une tâche

Objectif	Soumettre une tâche pour son exécution
Acteur(s)	Chercheur
Pré-condition	La tâche a été importée et le chercheur est authentifié
Scénario nominal	1. L'utilisateur sélectionne la tâche à soumettre et valide. 2. Le Master, grâce aux annotations, découpe et distribue la tâche à exécuter. 3. Les Workers exécutent la tâche et renvoient les résultats au Master pour recomposition. 4. Le Master recompose et renvoie le résultat à l'utilisateur.
Scénarios alternatifs	— A1 : Erreur de connexion — Un message d'erreur de connexion est renvoyé à l'utilisateur. — A2 : Dossier/fichier mal annoté — Le système renvoie un message d'erreur d'annotation.
Post-condition	Le résultat de l'exécution est renvoyé à l'utilisateur

TABLE 2.4 – Cas d'utilisation : Soumission d'une tâche

Objectif	Permettre à l'utilisateur de consulter les résultats de ses tâches exécutées
Acteur(s)	Chercheur
Pré-condition	Être authentifié
Scénario nominal	1. L'utilisateur sélectionne le menu des résultats. 2. Le système affiche les résultats et les détails d'exécution (durée, date, etc.).
Scénarios alternatifs	— A1 : Aucun résultat — Le système affiche un message indiquant l'absence de résultats. — A2 : Erreur de récupération — Un message d'erreur est renvoyé à l'utilisateur.
Post-condition	Les résultats sont affichés à l'utilisateur

TABLE 2.5 – Cas d'utilisation : Consultation des résultats

Objectif	Consulter l'état d'une machine du cluster
Acteur(s)	Gestionnaire du cluster
Pré-condition	Être authentifié
Scénario nominal	1. L'utilisateur sélectionne l'option de consultation des machines. 2. Le système affiche la liste des machines du cluster. 3. L'utilisateur sélectionne la machine souhaitée. 4. Le système affiche les détails de la machine sélectionnée.
Scénarios alternatifs	— A1 : Machine indisponible — Le système affiche un message d'erreur d'accessibilité. — A2 : Liste vide — Le système indique qu'aucune machine n'est disponible.
Post-condition	Les détails de la machine sont affichés

TABLE 2.6 – Cas d'utilisation : Consultation d'une machine

Objectif	Déconnecter une machine du cluster
Acteur(s)	Gestionnaire du cluster
Pré-condition	Être authentifié
Scénario nominal	1. L'utilisateur sélectionne la machine à arrêter dans la liste. 2. L'utilisateur déclenche la commande d'arrêt distant. 3. Le système demande à l'agent sur la machine d'exécuter la commande d'arrêt. 4. L'état de la machine est mis à jour sur l'interface pour confirmer l'arrêt.
Scénarios alternatifs	— A1 : Échec de communication — Le système affiche un message d'échec et propose de réessayer. — A2 : Refus de confirmation — L'utilisateur annule, la procédure est abandonnée sans modification. — A3 : Machine en cours d'exécution critique — Le système demande une confirmation supplémentaire ou bloque l'action.
Post-condition	La machine est arrêtée (déconnectée du cluster)

TABLE 2.7 – Cas d'utilisation : Suppression d'une machine du cluster

Objectif	Voir en temps réel l'état d'un programme en cours d'exécution
Acteur(s)	Gestionnaire du cluster
Pré-condition	Être authentifié
Scénario nominal	1. L'utilisateur sélectionne l'option de consultation des tâches en cours. 2. Le système affiche la liste des tâches actives. 3. L'utilisateur sélectionne la tâche souhaitée. 4. Le système affiche les détails d'exécution (machines impliquées, pourcentage d'avancement, etc.).
Scénarios alternatifs	— A1 : Aucune tâche en cours — Le système affiche un message d'absence de tâches actives. — A2 : Erreur de récupération — Le système affiche un message d'erreur et propose de réessayer.
Post-condition	Les détails d'exécution de la tâche sont affichés

TABLE 2.8 – Cas d'utilisation : Consultation d'un programme en cours d'exécution

2.1.3 Identification des agents

L'examen des cas d'utilisation révèle quatre entités autonomes aux responsabilités distinctes.

Groupe- ment fonctionnel	Justification d'autonomie	Agent
Accueillir les soumissions de code, exposer l'interface de supervision et relayer les programmes vers le Master	Point d'entrée autonome, indépendant du flux de calcul	«Agent» Réceptionniste
Gérer la réception, décomposition et recomposition des calculs	Vision globale du calcul et des ressources	«Agent» Master
Exécuter des calculs locaux	Autonomie d'exécution, capacité de refus selon charge	«Agent» Worker
Superviser l'état du réseau et détecter les pannes	Observation indépendante du flux de calcul	«Agent» Contrôleur

TABLE 2.9 – Identification des agents

L'Agent Réceptionniste constitue le point d'entrée unique du système : il reçoit les soumissions de code des utilisateurs, les relaie au Master et expose une interface de supervision du cluster. L'Agent Master assume les fonctions de décomposition des calculs, d'orchestration de leur répartition et d'agrégation des résultats. L'Agent Worker est responsable de l'exécution locale des sous-calculs qui lui sont confiés ; chaque machine physique héberge une instance de cet agent. L'Agent Contrôleur maintient une vision globale de l'état du système et détecte les anomalies.

2.1.4 Identification des rôles

Agent	Rôle	Responsabilité observable
Réceptionniste	Passerelle utilisateur Superviseur HTTP	Recevoir les soumissions de code et les relayer au Master Exposer l'état du cluster et les journaux d'exécution via une
Master	Réceptionniste Générateur Orchestracteur Synthétiseur	Accueillir et valider les programmes soumis Transformer un programme annoté en unités distribuables Décider de l'allocation des sous-tâches aux Workers Combiner les résultats partiels en résultat final
Worker	Exécuteur Signaleur d'état	Réaliser le calcul local et signaler la terminaison Émettre périodiquement les métriques de charge
Contrôleur	Observateur Détecteur de panne	Collecter et centraliser les informations d'état Identifier les anomalies et alerter les agents concernés

TABLE 2.10 – Identification des rôles

2.1.5 Identification du SMA

Organisation

L'organisation est hiérarchique fonctionnelle sans contrôle autoritaire. Le Master coordine mais ne commande pas : il propose des sous-calculs aux Workers qui conservent le droit de refuser. Le Contrôleur observe sans intervenir dans le flux de calcul. Cette organisation émerge des dépendances naturelles : le Master dépend des Workers pour l'exécution, les Workers dépendent du Master pour l'obtention des tâches, et le Master dépend du Contrôleur pour une vision fiable des ressources.

Tous les agents partagent le même objectif global — l'exécution efficace des calculs — mais avec des points de vue partiels. Le Master a une vue globale du calcul mais pas de l'état réseau en temps réel. Le Worker a une vue locale de sa charge mais ignore la charge des autres. Le Contrôleur a une vue globale de la présence des nœuds mais ignore la sémantique des calculs.

Environnement

L'environnement est un réseau local académique aux caractéristiques contraignantes. La couche physique comprend un switch Ethernet à cinq ports et six câbles RJ45 sertis manuellement, limitant l'extension à six nœuds. La latence est faible mais la bande passante est modeste. L'absence de redondance réseau fait du switch un point de défaillance unique.

La couche informatique est hétérogène : trois machines DELL sous Ubuntu Server 18.04 avec des processeurs Intel Core 2 Duo et des mémoires de 2 à 4 Go ; trois thin clients Wyse sous Windows CE avec 32 à 64 Mo de RAM et des processeurs de 533 MHz à 1 GHz.

Chaque agent perçoit son environnement via les métriques locales de sa machine ($LOAD_v$, NET_v) et via les messages reçus des autres agents. Il agit sur l'environnement en envoyant des messages TCP, en exécutant des calculs localement ou en modifiant l'état du cluster via les mécanismes d'élection et d'enregistrement.

L'environnement est dynamique : ajout et retrait volontaires par le Gestionnaire, pannes et surcharges imprévues, connectivité Internet externe instable rendant le cluster autonome une nécessité.

Séquences système

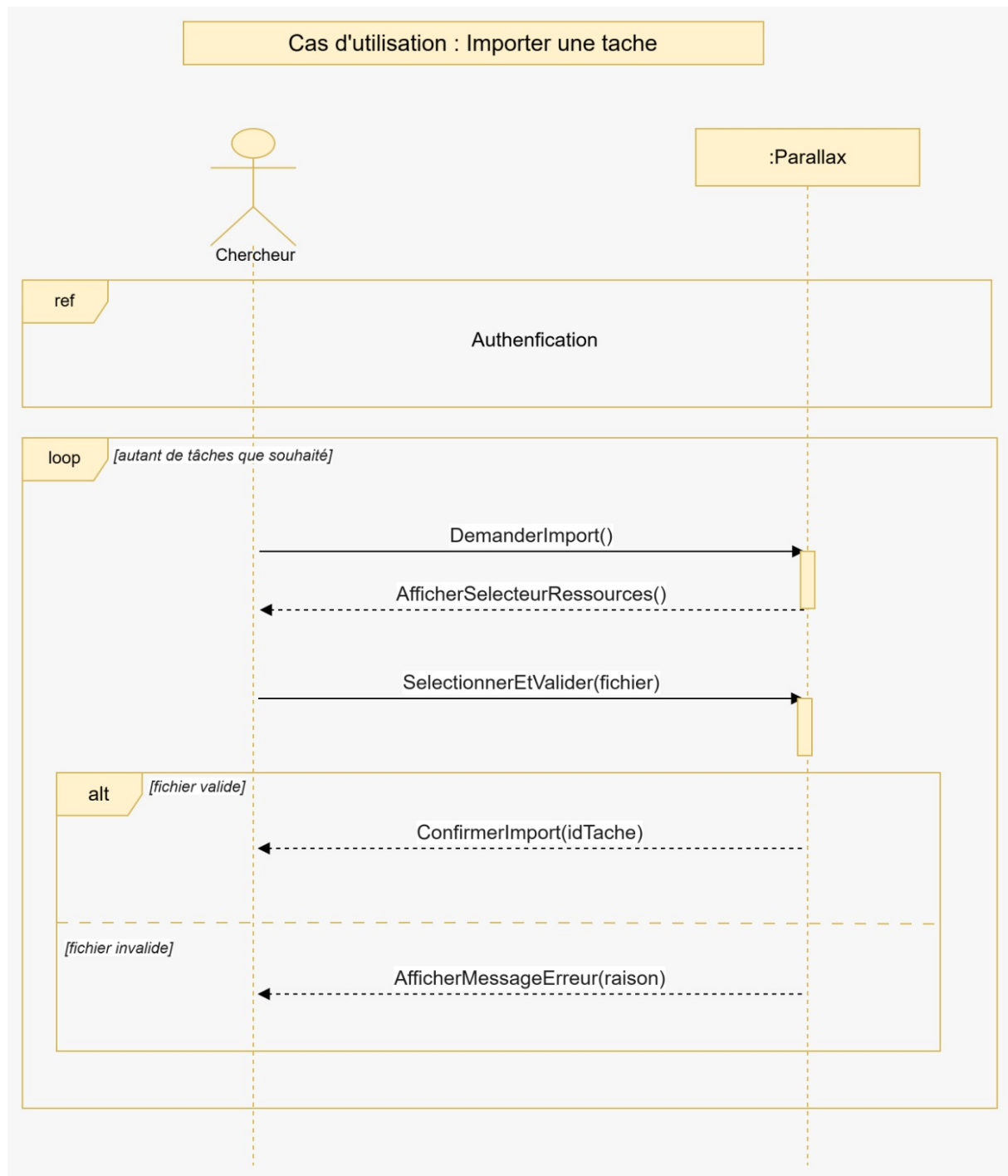


FIGURE 2.3 – Diagramme de séquences système — Importer une tâche

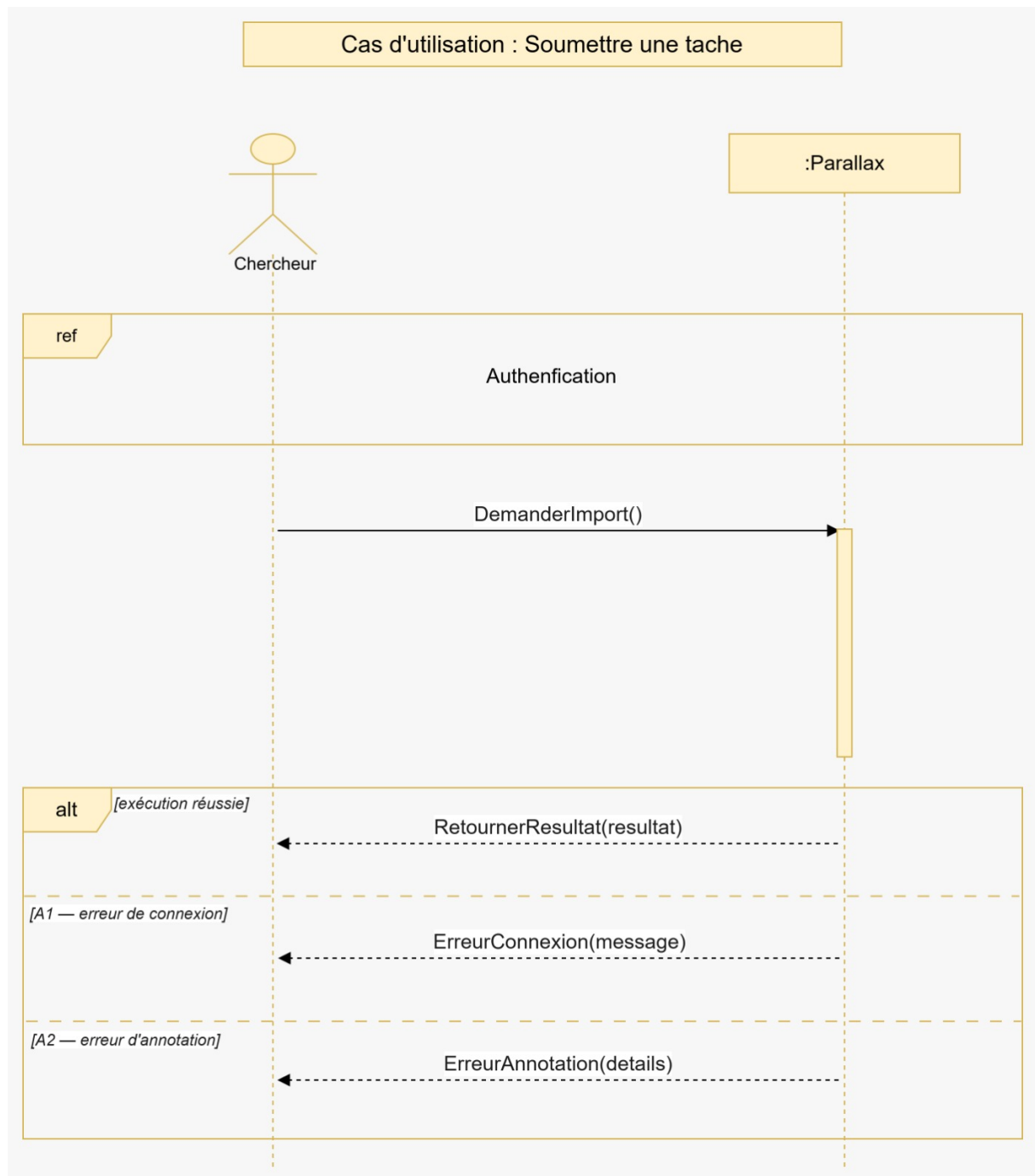


FIGURE 2.4 – Diagramme de séquences système — Soumettre une tâche

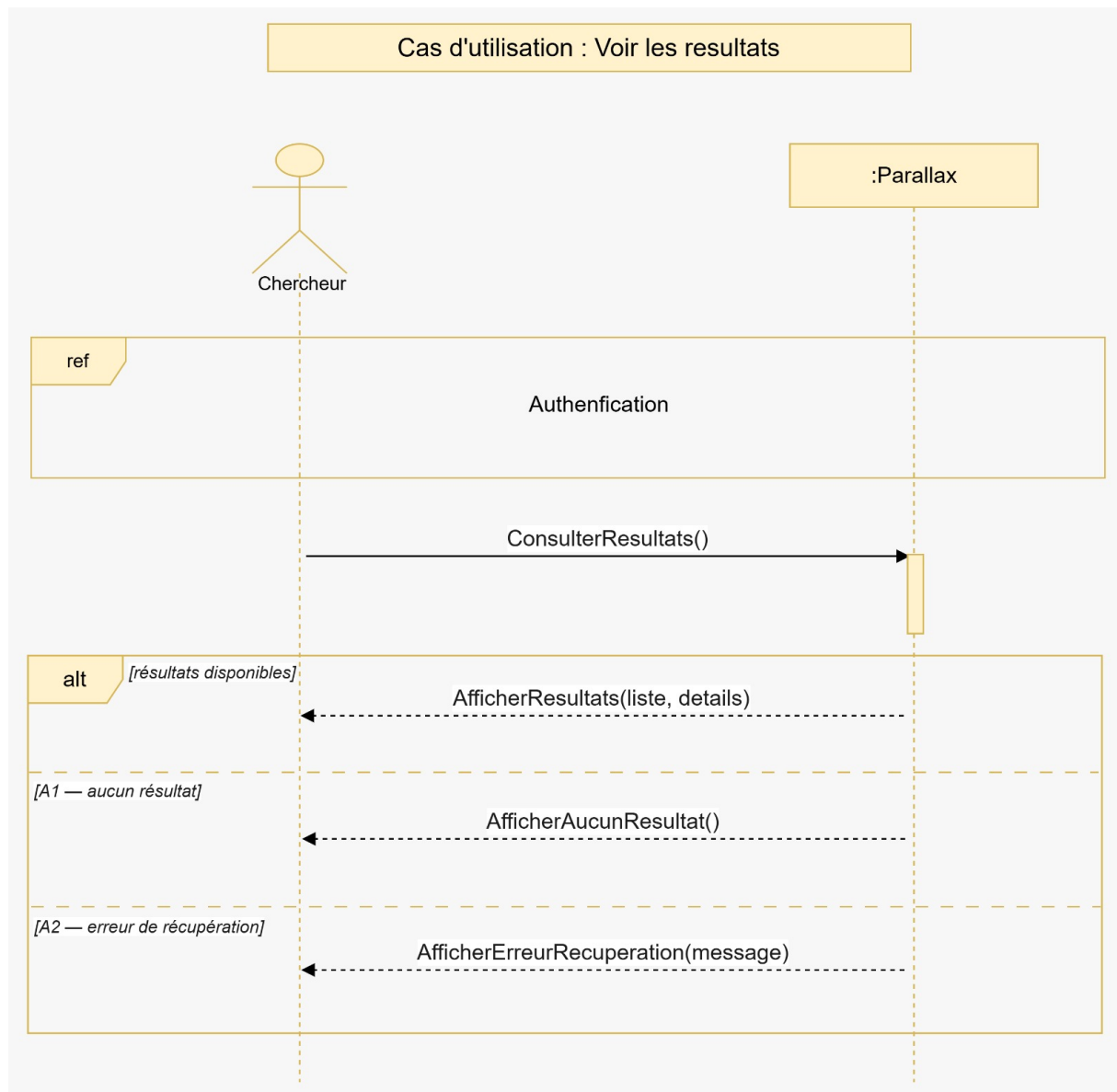


FIGURE 2.5 – Diagramme de séquences système — Voir les résultats

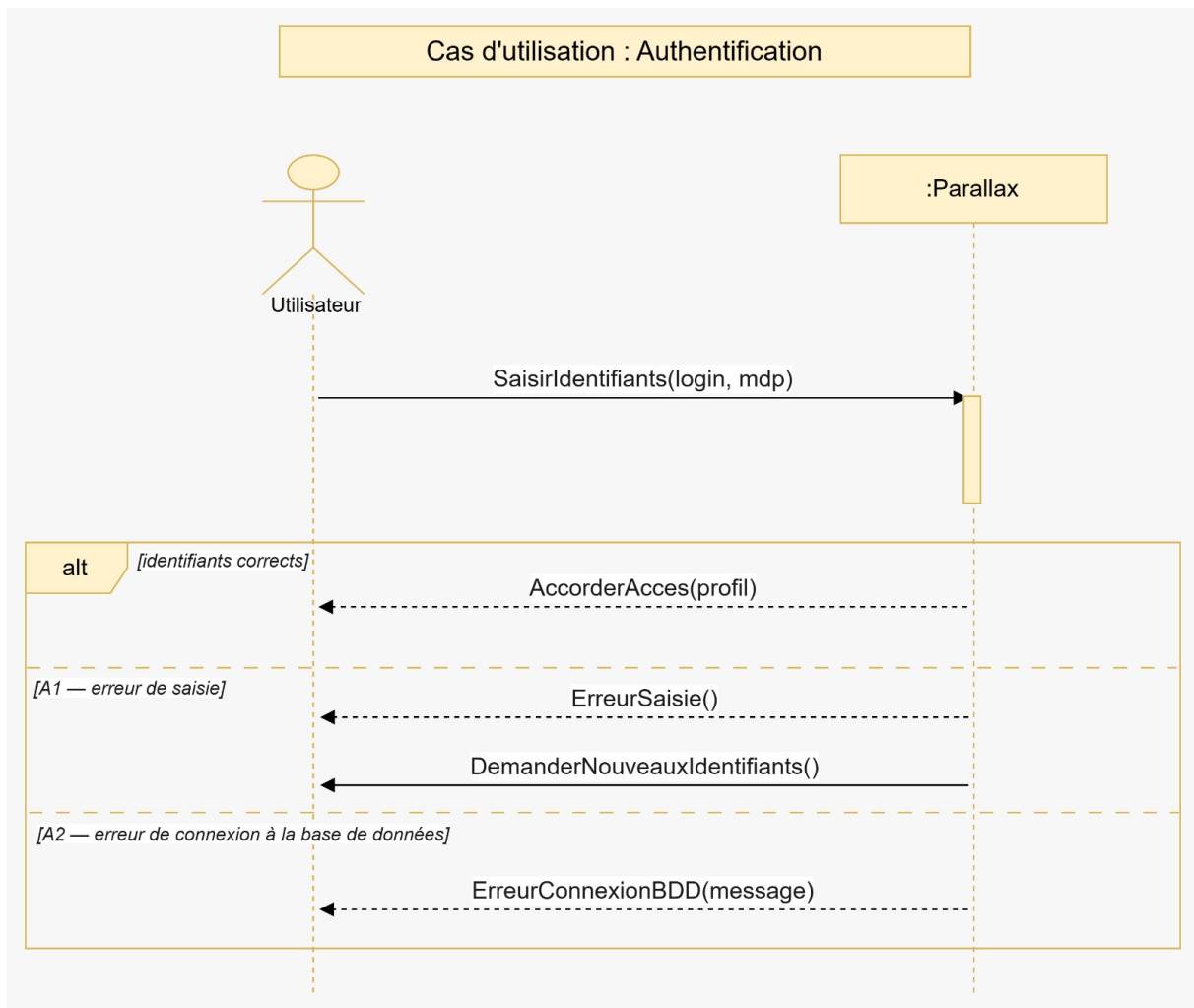


FIGURE 2.6 – Diagramme de séquences système — Authentification

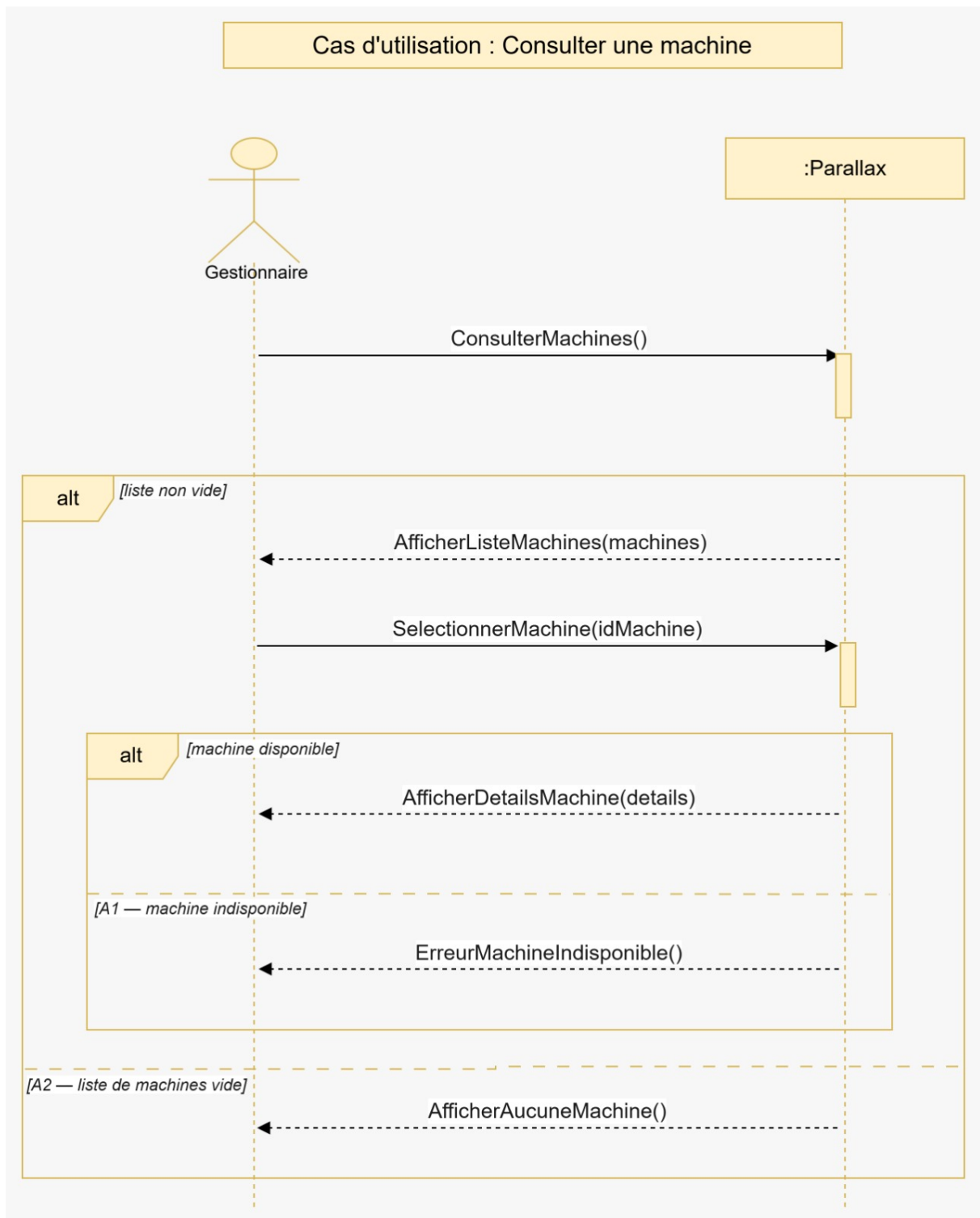


FIGURE 2.7 – Diagramme de séquences système — Consulter une machine

2.2 Conception

La conception du système traduit les éléments identifiés en phase d'analyse en une architecture opérationnelle. Elle définit la structure des agents, leurs comportements, les

protocoles de communication et les mécanismes de coordination distribuée assurant l'exécution des calculs dans un cluster hétérogène.

2.2.1 Architecture du SMA

L'architecture de SMA_PARALLAX est hiérarchique fonctionnelle à trois niveaux :

- **Niveau supervision** (Contrôleur) : maintient une vue globale cohérente, détecte les pannes, gère l'élection.
- **Niveau coordination** (Master) : reçoit les calculs, décompose, distribue et agrège les résultats.
- **Niveau exécution** (Workers) : réalise les sous-calculs de manière autonome et signale leur état.

Cette architecture hybride combine la réactivité du niveau inférieur (heartbeat, détection rapide) et la délibération du niveau supérieur (planification de la distribution, élection du remplaçant).

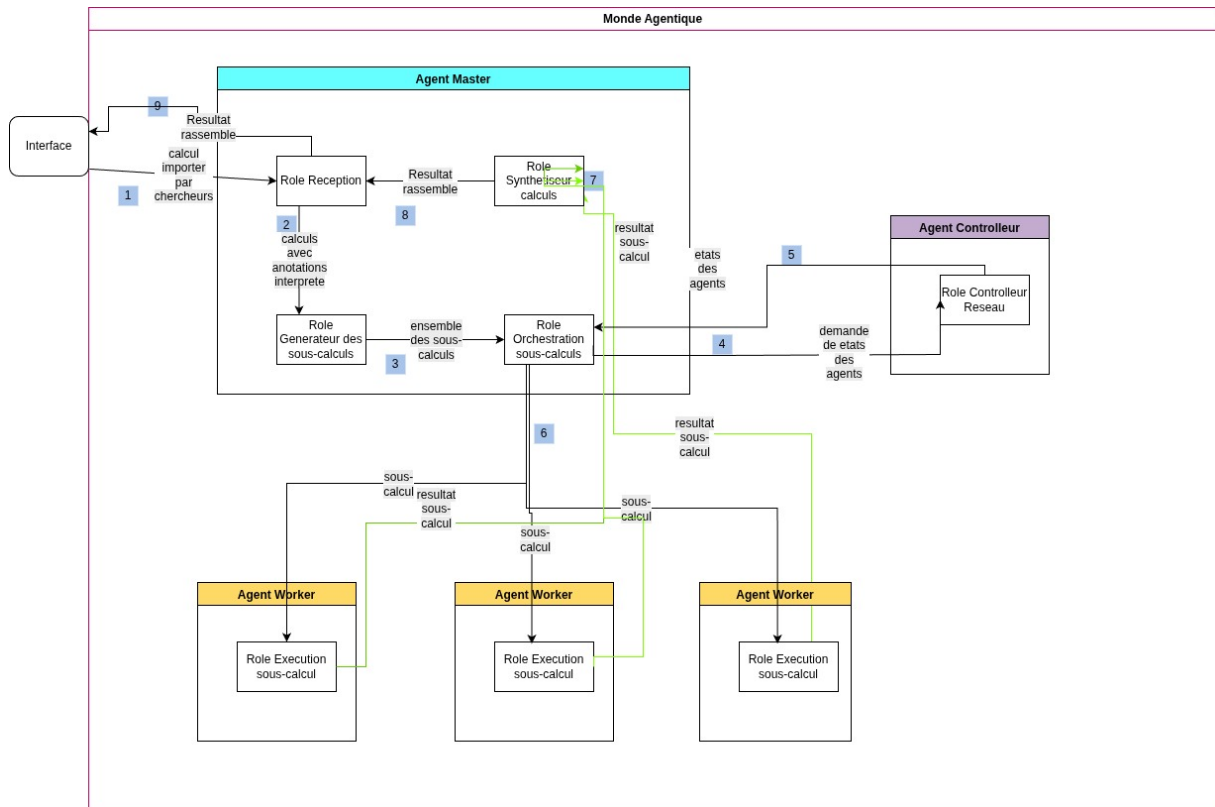


FIGURE 2.8 – Architecture agentique de SMA_PARALLAX

2.2.2 Structure de l'agent et du SMA

Modélisation physique d'un nœud

Chaque machine $v \in V_M$ est caractérisée par un vecteur d'état :

$$X_v = \begin{bmatrix} ID_v \\ NET_v \\ COMPUTE_v \\ MEMORY_v \\ STORAGE_v \\ LOAD_v \\ RELIABILITY_v \\ STATE_v \\ SECURITY_v \end{bmatrix}$$

- * ID_v = (uuid, role, region) : identité unique, rôle et localisation.
- * NET_v = (ip, latency, bandwidth_{in}, bandwidth_{out}, jitter, loss) : performances réseau.
- * $COMPUTE_v$ = (cpucore, cpufreq, gpu, flops) : capacités de calcul.
- * $MEMORY_v$ = (ram, mem_bandwidth) : ressources mémoire.
- * $STORAGE_v$ = (disk, io) : capacités de stockage.
- * $LOAD_v$ = (cpuusage, ramusage, queue) : charge courante.
- * $STATE_v$ = (status, availability) : état opérationnel.
- * $RELIABILITY_v$ = (uptime, failurerate) : fiabilité.
- * $SECURITY_v$ = (auth, trust) : sécurité et confiance.

Modélisation mentale : architecture BDI des agents cognitifs

Les agents Master et Contrôleur suivent une architecture BDI (Belief-Desire-Intention), fondée sur la théorie de l'action rationnelle de Michael Bratman telle que présentée dans le cours [9], qui structure leur état mental en trois composantes [4] :

- **Croyances** (B) : représentation informationnelle de l'état du monde. Le Master maintient des croyances sur l'état des Workers (charge, disponibilité) via le Contrôleur. Le Contrôleur maintient des croyances sur la présence et la santé de tous les nœuds via les heartbeats.
- **Désirs** (D) : états que l'agent souhaite atteindre. Le Master désire minimiser le temps global d'exécution des calculs. Le Contrôleur désire maximiser la disponibilité du cluster.
- **Intentions** (I) : engagements délibérés que l'agent poursuit activement. Le Master forme l'intention de distribuer les sous-calculs selon le score pondéré. Le Contrôleur forme l'intention de déclencher une élection lors d'une défaillance critique.

Par exemple, lorsque le Contrôleur envoie une alerte de panne d'un Worker en cours d'exécution, le Master révisé ses croyances (ce Worker n'est plus disponible), met à jour ses intentions (réaffecter les tâches en cours de ce Worker) et planifie immédiatement leur redistribution vers les nœuds disponibles restants.

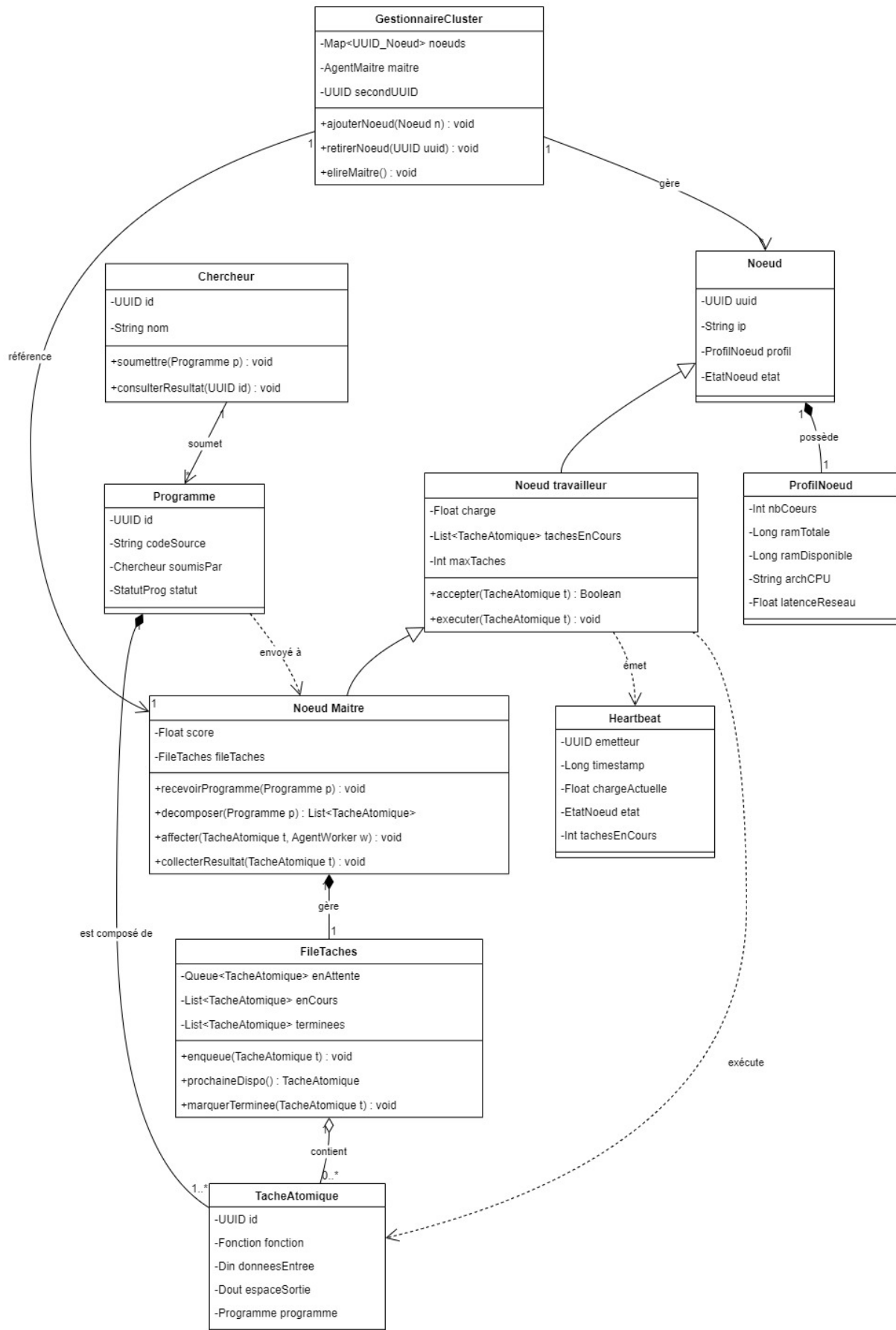
Les agents Worker et Réceptionniste, de type réactif avec état, ne suivent pas le cycle BDI complet : ils maintiennent un état interne minimal (charge pour le Worker, coordonnées du Master/-Contrôleur pour le Réceptionniste) et réagissent par des règles condition-action sans délibération symbolique.

Typologie des agents de SMA_PARALLAX

Agent	Type	Justification	État interne
AgentRéceptionniste	Réactif avec état [9]	Perçoit les requêtes HTTP et les soumissions utilisateur, met en cache les programmes et journaux, réagit par transfert réseau conditionnel	UUID, IP du Contrôleur, IP du Master actif, cache des logs locaux
AgentWorker	Réactif avec état [9]	Perçoit l'environnement via heartbeat, maintient un état interne (charge, file de tâches), réagit par règles condition-action sans délibération	$LOAD_v$, file des tâches en cours
AgentMaster	Cognitif orienté but [9]	Représente symboliquement l'état du cluster, planifie la décomposition et la distribution selon un but de minimisation du temps global	Croyances sur le cluster, buts de répartition, intentions de distribution
AgentContrôleur	Cognitif orienté utilité [9]	Évalue les configurations du cluster via une fonction d'utilité (fiabilité, cohérence de la vue globale)	Modèle social des nœuds, historique des pannes

TABLE 2.11 – Typologie des agents de SMA_PARALLAX

Diagramme de classes



Actions et événements du système

N°	Action	Commentaire	Rôle	Entrée	Sortie
1	Réception des calculs	Prise en charge des requêtes	Réception (Master)	Requête utilisateur	Tâche globale
2	Décomposition	Division en sous-calculs	Génération (Master)	Tâche globale	Sous-calculs
3	Distribution	Allocation aux Workers	Orchestration (Master)	Sous-calculs + états	Tâches assignées
4	Réception sous-calcul	Réception par le Worker	Exécution (Worker)	Sous-calcul	Tâche locale
5	Exécution	Calcul effectif	Exécution (Worker)	Tâche locale	Résultat partiel
6	Retour résultat	Envoi au Master	Exécution (Worker)	Résultat partiel	Résultat transmis
7	Agrégation	Combinaison des résultats	Synthèse (Master)	Résultats partiels	Résultat final
8	Observation	Surveillance ressources	Contrôle (Master)	États agents	Vue système
9	Diffusion état Worker	Communication charge/dispo	Contrôle (Worker)	État local	Infos d'état
10	Collecte états	Centralisation des infos	Contrôle (Contrôleur)	Infos d'état	BDD état
11	Détection pannes	Identification anomalies	Contrôle (Contrôleur)	États système	Alertes
12	Notification défaillance	Alerte au Master	Contrôle (Contrôleur)	Alertes	Notification
13	Enregistrement nœud	Ajout au cluster	Contrôle (Contrôleur)	Infos nœud	Nœud enregistré

TABLE 2.12 – Table des actions de SMA_PARALLAX

N°	Événement	Description
E1	Soumission de calcul	Un utilisateur soumet une requête via l'interface. Déclenche l'agent de réception.
E2	Réception tâche globale	Le Master reçoit et valide la tâche.
E3	Décomposition déclenchée	La tâche est transformée en sous-calculs.
E4	Disponibilité des ressources	Les états des Workers orientent la planification.
E5	Allocation des sous-calculs	Les sous-tâches sont assignées selon le score pondéré.
E6	Réception sous-calcul	Un Worker reçoit une tâche à exécuter.
E7	Début d'exécution	Le Worker commence le traitement.
E8	Fin d'exécution	Le Worker produit un résultat partiel.
E9	Envoi résultat partiel	Le résultat est transmis au Master.
E10	Réception résultats partiels	Le Master collecte les résultats des Workers.
E11	Agrégation	Les résultats partiels sont combinés.
E12	Production résultat final	Le résultat est prêt à être renvoyé à l'utilisateur.
E13	Diffusion état Worker	Les Workers envoient périodiquement leur état.
E14	Collecte des états	Le Contrôleur centralise les informations.
E15	Détection de défaillance	Une anomalie est détectée.
E16	Notification de défaillance	Une alerte est envoyée au Master.
E17	Enregistrement nouveau nœud	Un nouvel agent rejoint le système.

TABLE 2.13 – Table des événements de SMA_PARALLAX

2.2.3 Comportement des agents

Le comportement des agents est régi par leurs états internes et les transitions déclenchées par les événements du système.

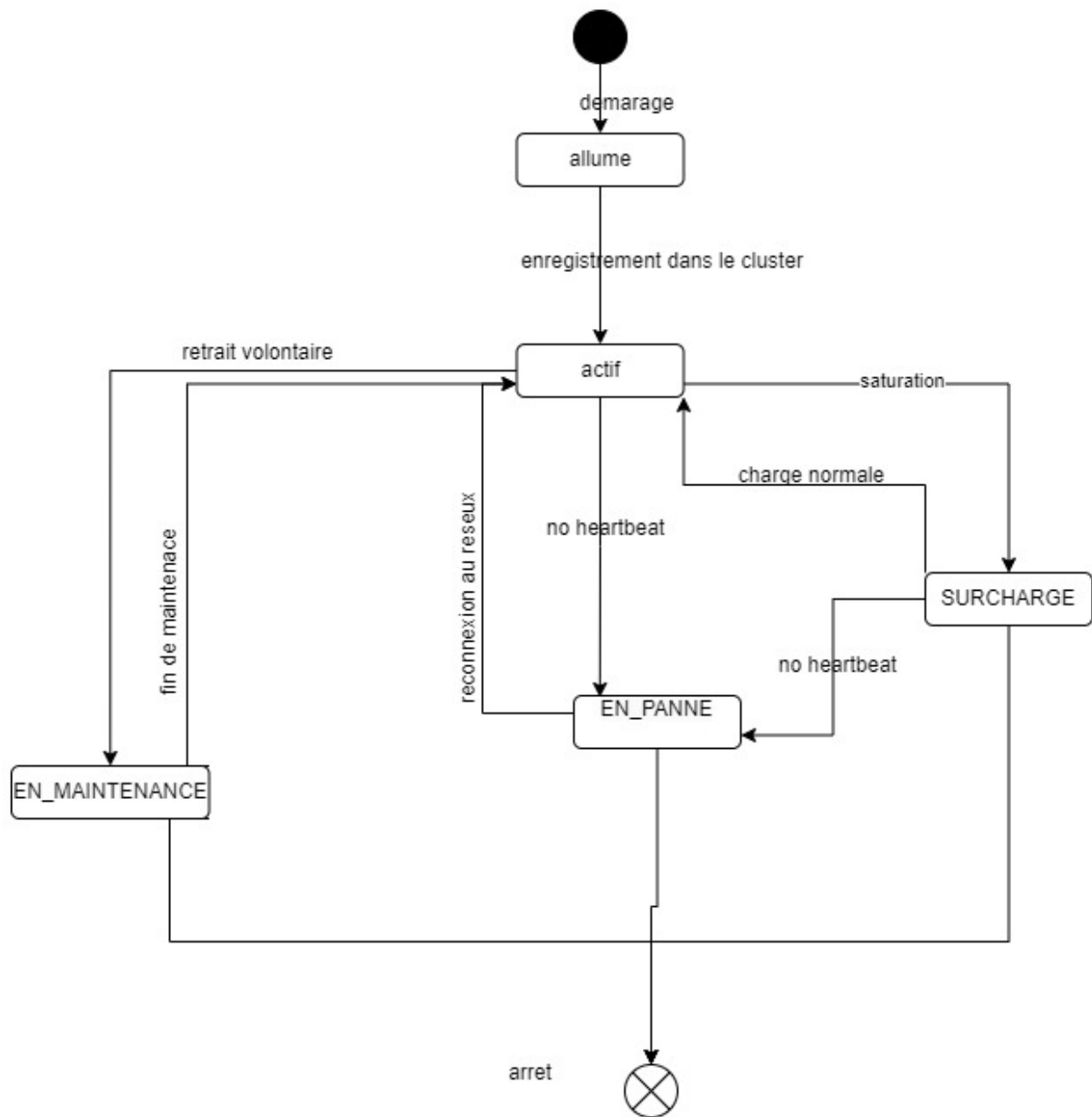


FIGURE 2.10 – Diagramme d'état d'un nœud

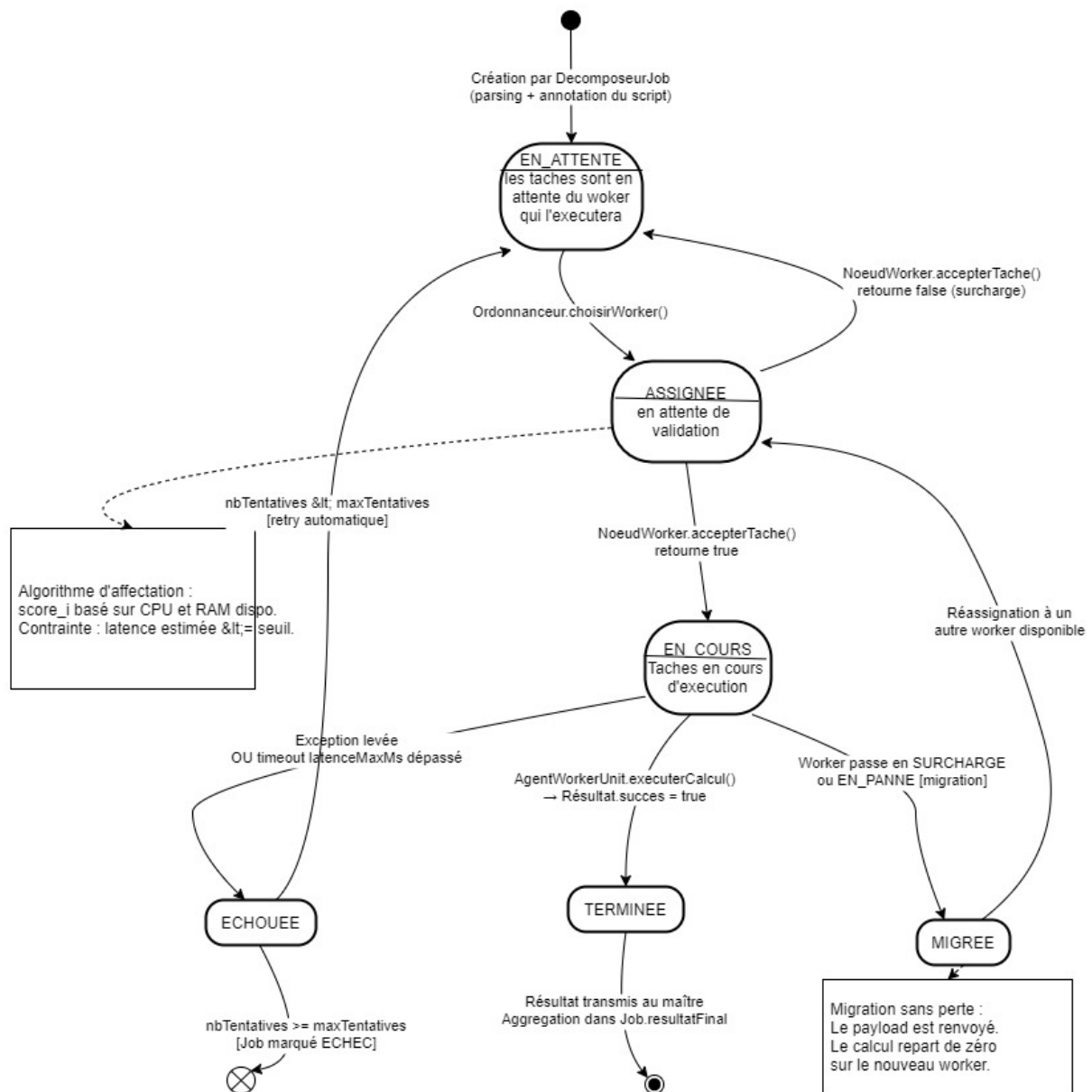


FIGURE 2.11 – Diagramme d'état d'une tâche

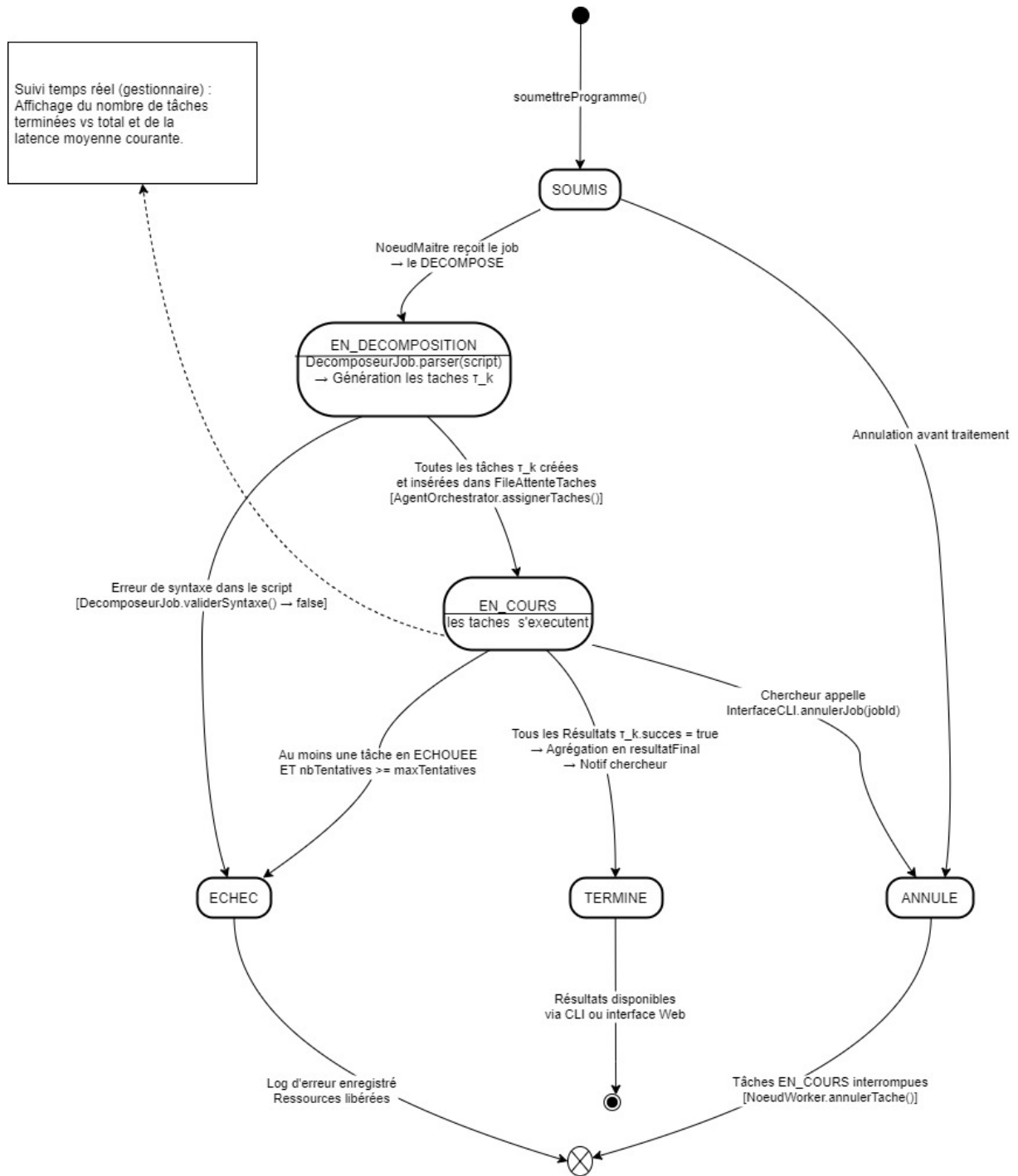


FIGURE 2.12 – Diagramme d'état d'un programme

2.2.4 Description des rôles

Les diagrammes suivants illustrent les interactions conçues entre les rôles identifiés en analyse.

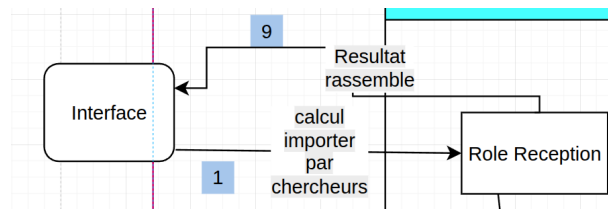


FIGURE 2.13 – Interaction entre le rôle de réception et son environnement

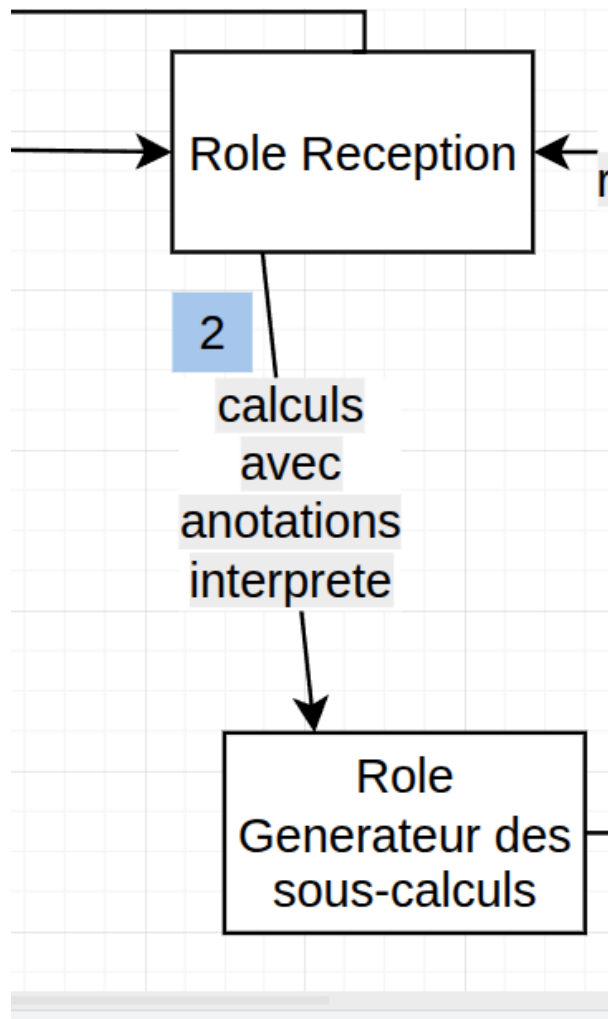


FIGURE 2.14 – Interaction entre le réceptionniste et le générateur de sous-tâches

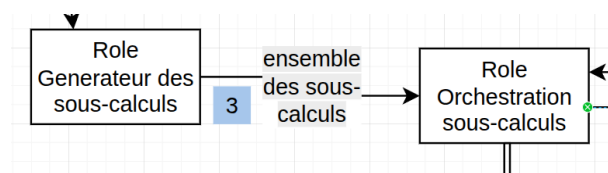


FIGURE 2.15 – Interaction détaillée entre réception et génération des sous-calculs

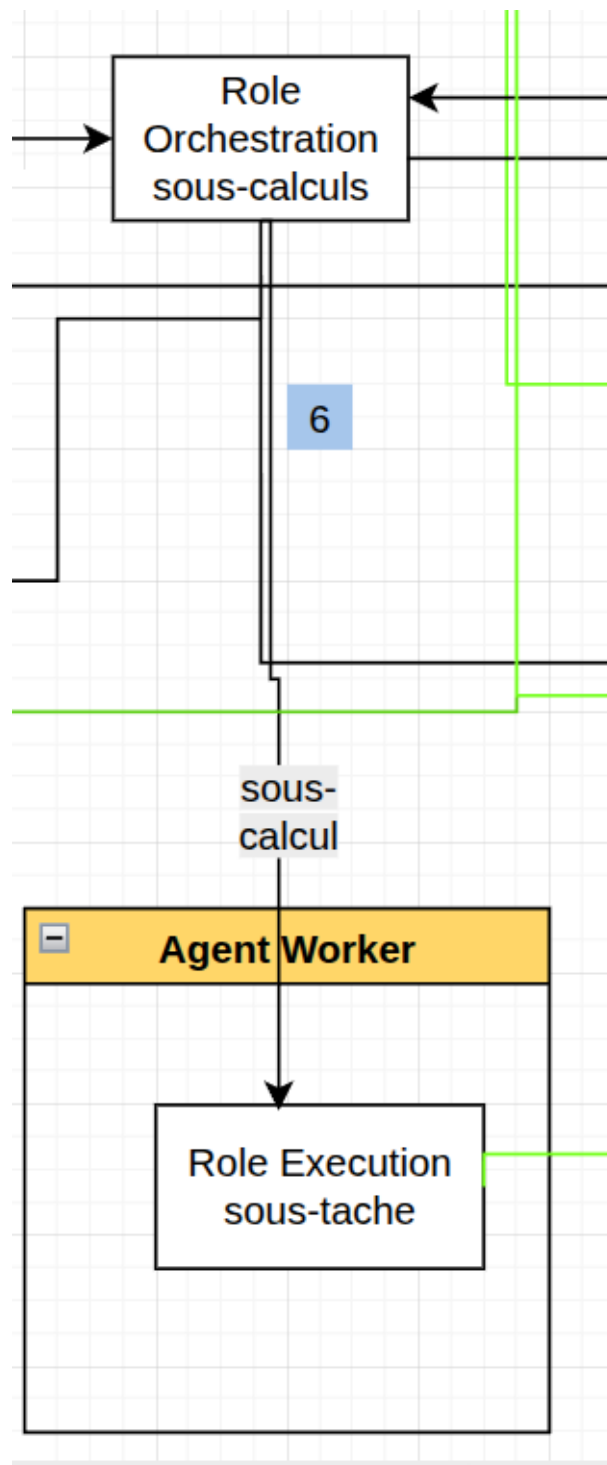


FIGURE 2.16 – Interaction entre le générateur de sous-calculs et l'orchestrateur

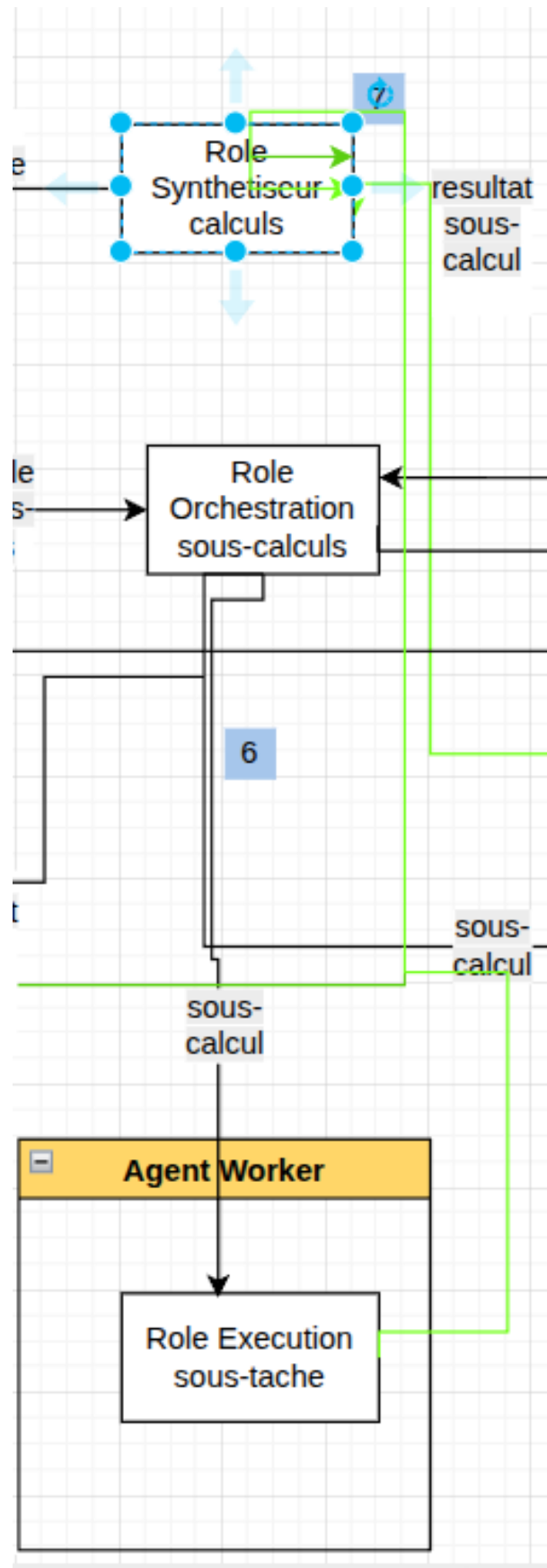


FIGURE 2.17 – Interaction entre les agents d'exécution et le synthétiseur

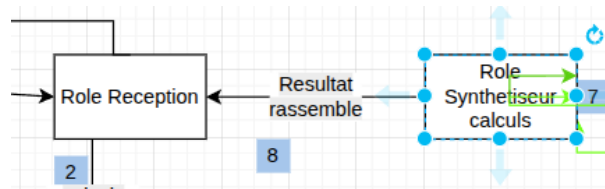


FIGURE 2.18 – Interaction entre le synthétiseur et le réceptionniste

Les diagrammes de séquences techniques suivants décrivent le comportement détaillé de chaque rôle au niveau de l'implémentation.

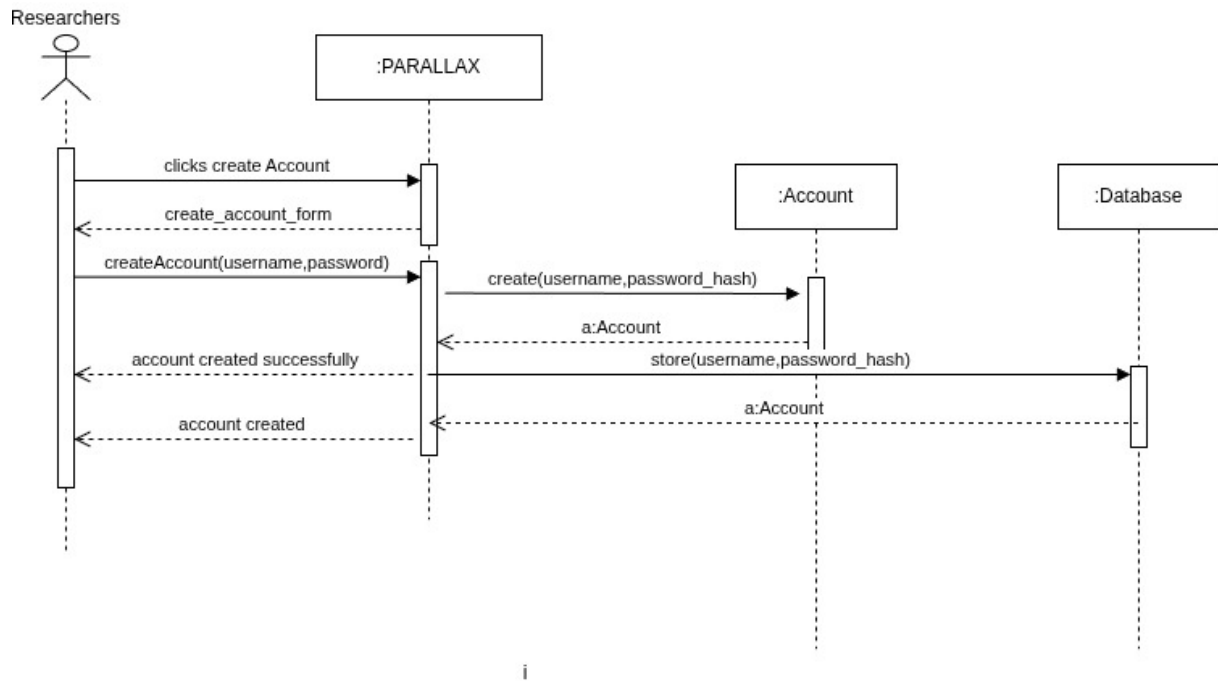


FIGURE 2.19 – Diagramme de séquences technique — Authentification

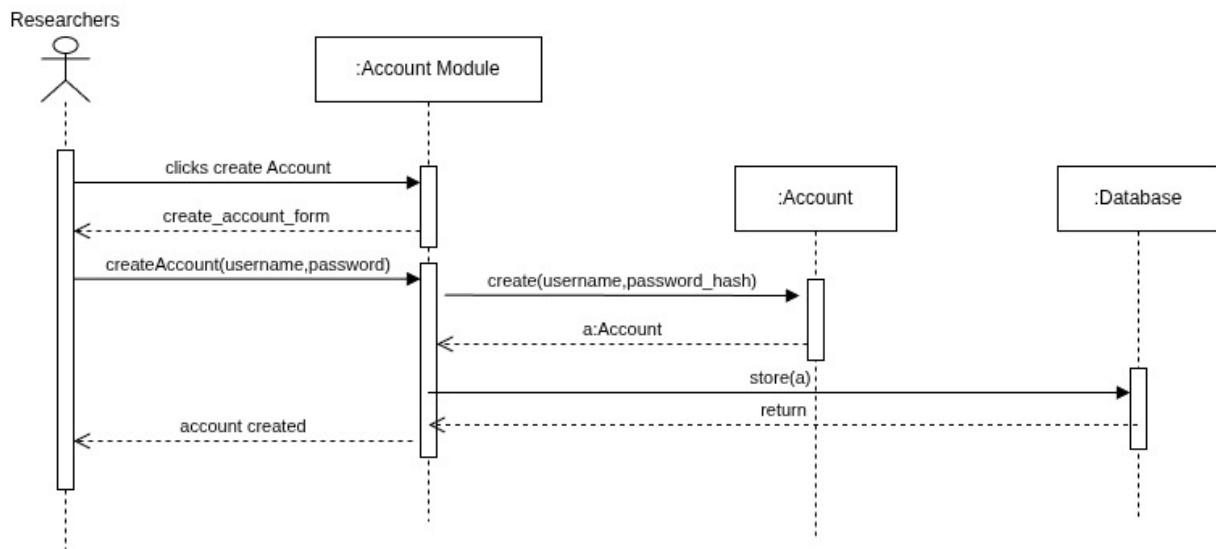


FIGURE 2.20 – Diagramme de séquences technique — Création de compte

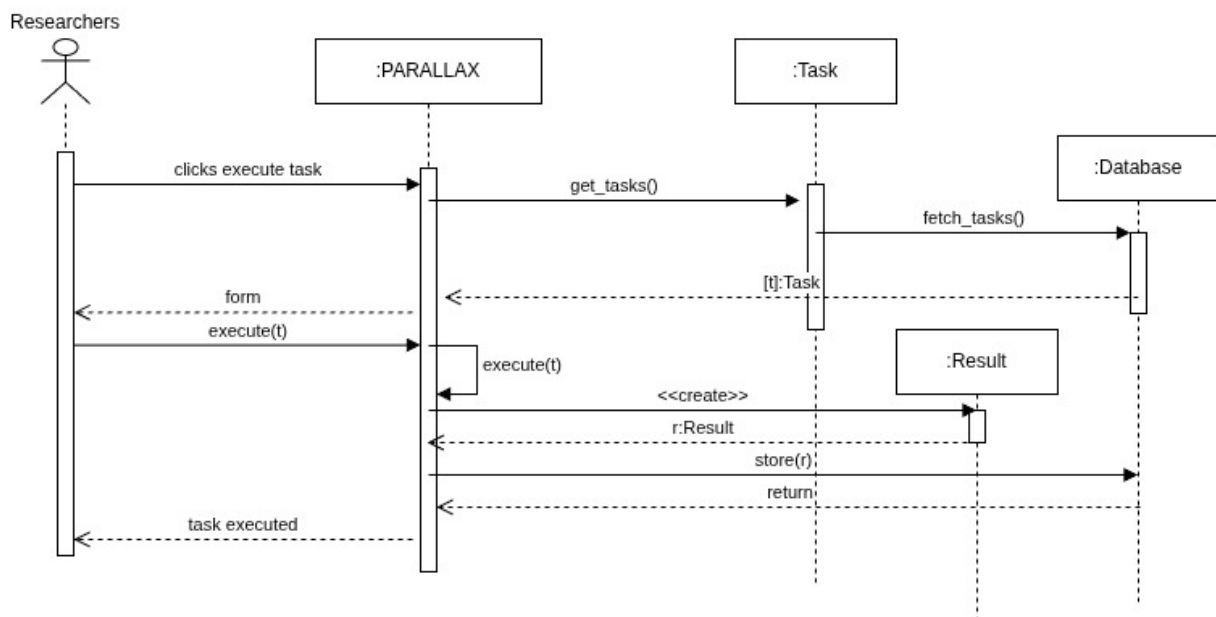


FIGURE 2.21 – Diagramme de séquences technique — Importation d'une tâche

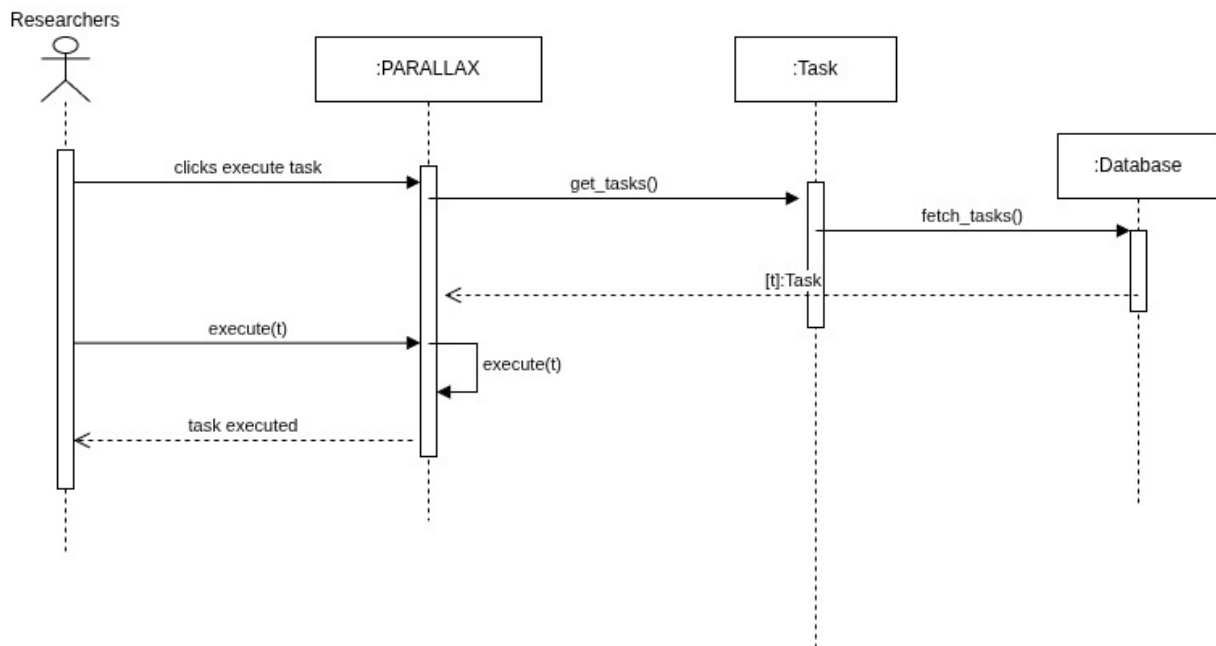


FIGURE 2.22 – Diagramme de séquences technique — Exécution d’une tâche

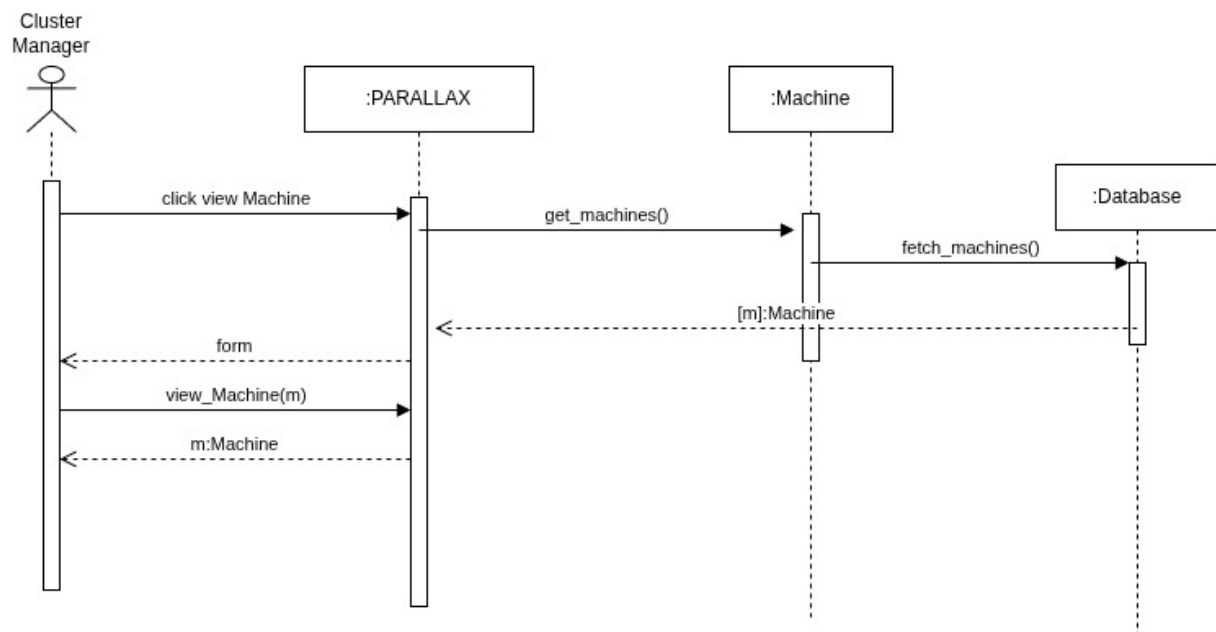


FIGURE 2.23 – Diagramme de séquences technique — Consultation des machines

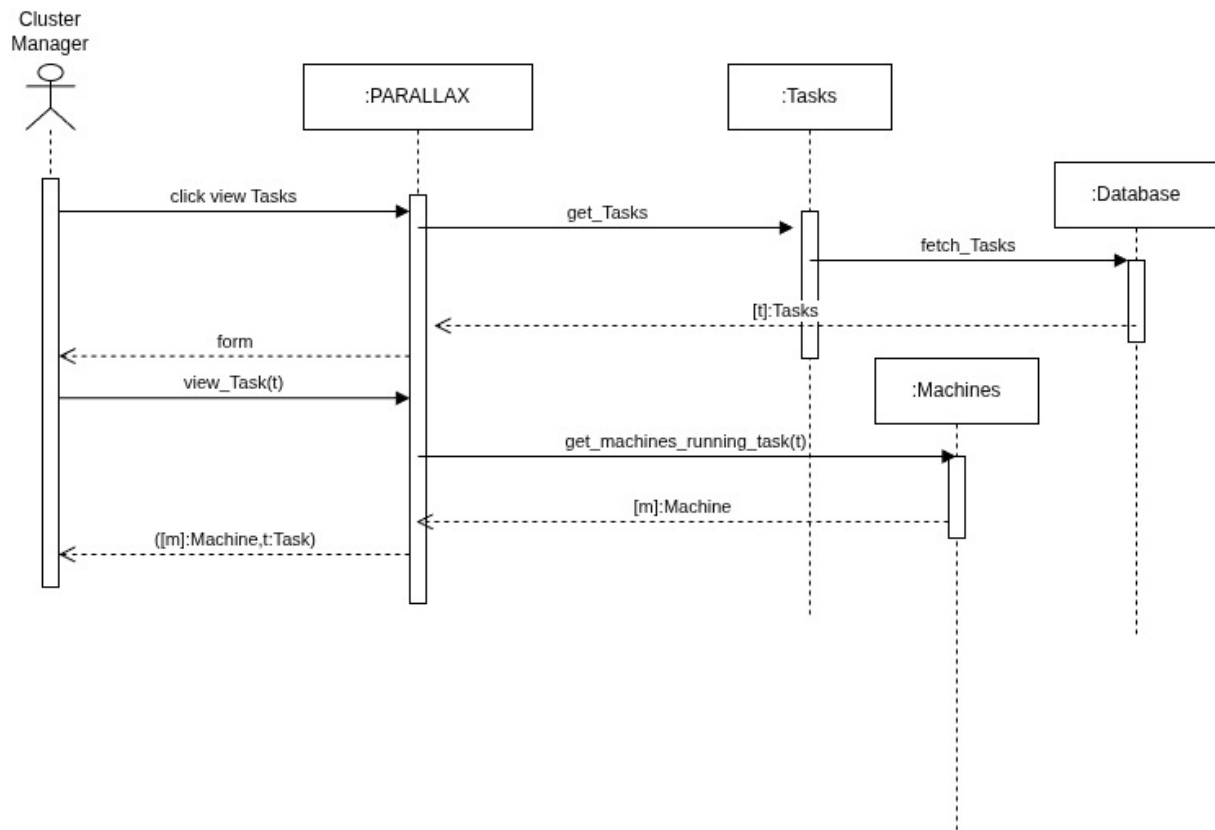


FIGURE 2.24 – Diagramme de séquences technique — Consultation des tâches

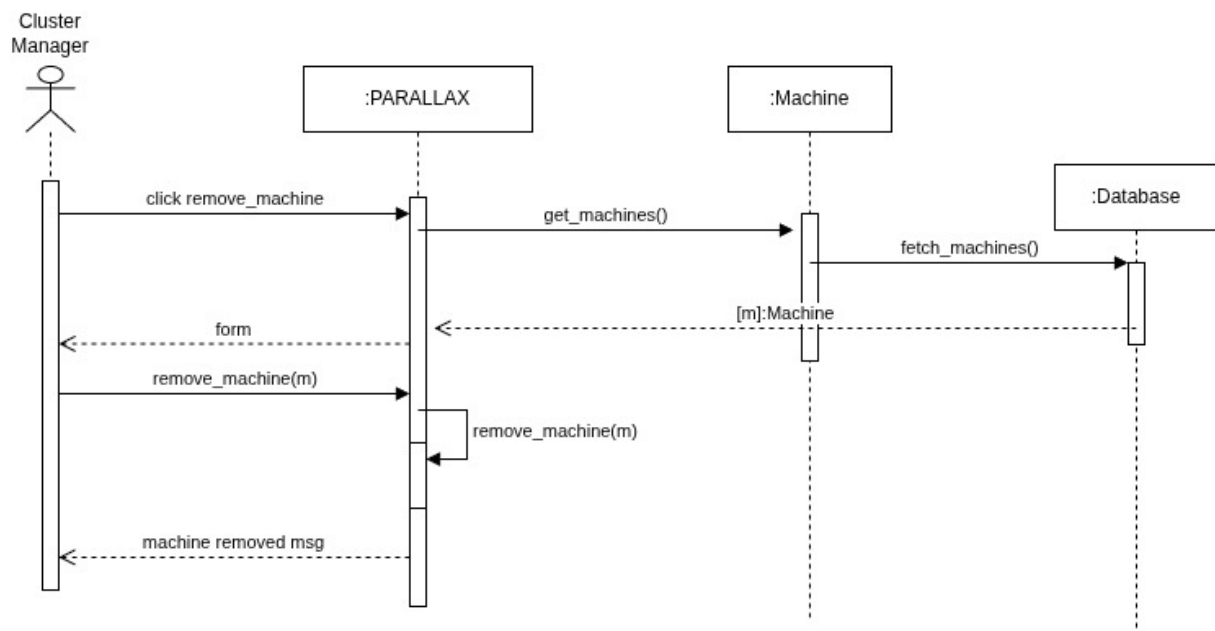


FIGURE 2.25 – Diagramme de séquences technique — Suppression d'un nœud

2.2.5 Protocoles de communication et coordination

Communication inter-agents

Dans SMA_PARALLAX, la communication entre agents est **directe et asynchrone**, reposant sur des messages structurés transportés via TCP sur le réseau local. Chaque message est un acte de communication au sens des ACL : il porte un type d'acte décrivant l'intention de l'émetteur (informer, demander, notifier), un émetteur, un récepteur, un contenu et un identifiant de conversation.

Type d'acte	Acte de communication	Direction	Fonction
Assertif	<i>inform</i>	Worker $\rightarrow$ Contrôleur	Signalement d'état de charge vi
Directif	<i>request</i>	Master $\rightarrow$ Worker	Demande d'exécution d'un sous
Notification	<i>notify</i>	Contrôleur $\rightarrow$ Master	Alerte de défaillance détectée

TABLE 2.14 – Types de messages dans SMA_PARALLAX

Structure d'un message de heartbeat

$$\text{HB}(i, t) = (\text{uuid}_i, k_i(t), \widehat{\text{CPU}}_i(t), \widehat{\text{RAM}}_i(t), q_i(t), \text{Score}(i, t), \sigma_i, L_i(t))$$

Chaque nœud n_i émet un heartbeat unicast vers le Contrôleur toutes les $T_{\text{HB}} = 2\text{s}$. Il transporte la télémétrie nécessaire à l'orchestration : charge CPU et RAM, longueur de la file de tâches, score courant et état logique.

Architecture à deux couches **Couche 1 — heartbeat unicast vers le Contrôleur.** Signal périodique de présence et de télémétrie sur le canal best-effort.

Couche 2 — gossip inter-pairs. Toutes les $T_G = 5\text{s}$, chaque nœud envoie à $k = \lceil \log_2 N \rceil$ pairs aléatoires un résumé de sa vue récente du cluster.

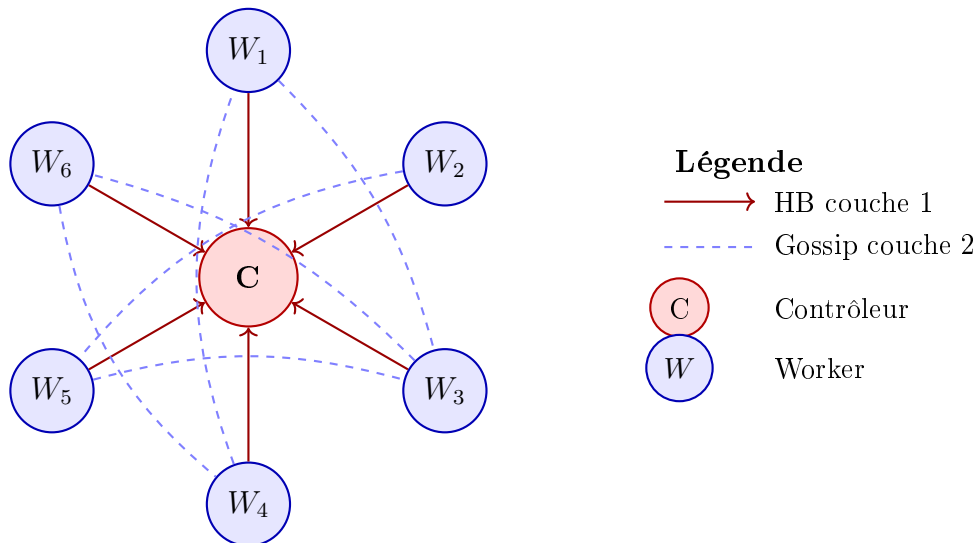


FIGURE 2.26 – Architecture à deux couches du heartbeat

Coordination distribuée : élection du nœud remplaçant

Objectif et déclencheurs Le protocole d'élection garantit la présence permanente d'un nœud **remplaçant** capable d'assurer la continuité en cas de défaillance d'un rôle critique. Trois événements déclenchent une élection :

- **(D1) Démarrage à froid** : aucun remplaçant n'est présent dans le cluster.
- **(D2) Remplacement de rôle critique** : le Master ou le Contrôleur tombe en panne.
- **(D3) Stepdowndu remplaçant** : maintenance, surcharge persistante ou réorganisation.

Fonction de score

$$score(n_i) = a \cdot \frac{CPU_{libre} \cdot CPU_{total}}{CPU_{libre}^{max} \cdot CPU_{total}^{max}} + b \cdot \frac{RAM_{libre} \cdot RAM_{total}}{RAM_{libre}^{max} \cdot RAM_{total}^{max}} + c \cdot \frac{LATENCE_{min}}{LATENCE_i} + d \cdot \frac{freq_i}{freq_{max}}$$

avec $a = 0,35$, $b = 0,35$, $c = 0,15$, $d = 0,15$.

Algorithm 1 Initiation d'une élection par le nœud k

```

1:  $\sigma_k \leftarrow electing$ 
2:  $H \leftarrow \{j \in V \mid Score_{j,t} > Score_{k,t}\}$ 
3: if  $H = \emptyset$  then
4:   diffuser  $coord(uuid_k)$ 
5:    $\sigma_k \leftarrow leader$ 
6: else
7:   for all  $j \in H$  do
8:     envoyer  $election(Score_{k,t}, uuid_k)$  à  $j$ 
9:   end for
10:  armer temporisation  $T_{attente}$ 
11: end if
```

Algorithm 2 Traitement des messages d'élection

```

1: procedure ONRECEIVEELECTION( $s, u$ )
2:   if  $Score_{k,t} > s$  ou ( $Score_{k,t} = s$  et  $uuid_k > u$ ) then
3:     envoyer  $alive$  à l'émetteur
4:     INITIAETELECTION( $k$ )
5:   end if
6: end procedure
7: procedure ONRECEIVECOORD( $u$ )
8:    $leader_k \leftarrow u$ 
9:    $\sigma_k \leftarrow normal$ 
10: end procedure
```

Coordination distribuée : détection de panne

Phase 1 — Suspicion. Si $\Delta_{ij}(t) \geq T_{suspect}$, le nœud i marque j comme SUSPECTED et émet un GOSSIP_QUERY à ses k voisins.

Phase 2 — Défaillance confirmée. Si $\Delta_{ij}(t) \geq T_{\text{failed}}$ et si la majorité des réponses sont négatives, i marque j comme FAILED et déclenche une élection si j était le Master.

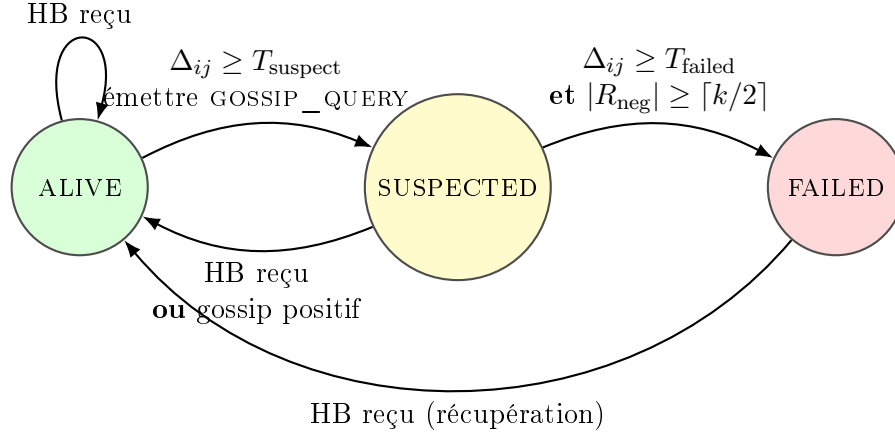


FIGURE 2.27 – Machine à états — suivi d’un pair par un observateur

Coopération dans l’exécution

Algorithm 3 Entrée d’un nœud dans le réseau

- 1: $N.ip \leftarrow \text{DHCP_REQUEST}()$
 - 2: $N.uuid \leftarrow \text{GENERATE_UUID}()$
 - 3: $\text{STORE_LOCAL}(N.uuid)$
 - 4: $\text{SEND_BROADCAST}(\text{HELLO}, N.uuid, N.ip)$
 - 5: **if** le Contrôleur reçoit HELLO **then**
 - 6: $\text{REGISTER_NODE}(N.uuid, N.ip)$
 - 7: $\text{SEND_ACK}(N.uuid)$
 - 8: **end if**
 - 9: $\text{CONNECT}(N, \text{CONTROLLER})$
-

Inscription d’un nœud

Algorithm 4 Processus de décomposition

- 1: Parser le programme et remplacer les annotations par les fonctions auxiliaires
 - 2: Exécuter séquentiellement les sections non distribuées
 - 3: **for** chaque section S distribuée **do**
 - 4: Interroger le Contrôleur pour l’état des nœuds disponibles
 - 5: Calculer la répartition : $w_i = x_i / \sum_j x_j$
 - 6: Envoyer le programme instrumenté aux Workers sélectionnés
 - 7: Envoyer à chaque Worker le couple $(\text{annotation_id}, \text{data})$
 - 8: **end for**
 - 9: Attendre tous les résultats (futex ou équivalent)
-

Décomposition et distribution des calculs

Algorithm 5 Processus de recomposition

- 1: Réceptionner les résultats partiels des Workers
 - 2: Mettre à jour le compteur de complétion via futex
 - 3: **if** tous les résultats attendus sont reçus **then**
 - 4: Appliquer la fonction **aggregate** définie par l'utilisateur
 - 5: Combiner les résultats partiels pour produire le résultat global
 - 6: **else**
 - 7: Attendre les résultats manquants
 - 8: **end if**
-

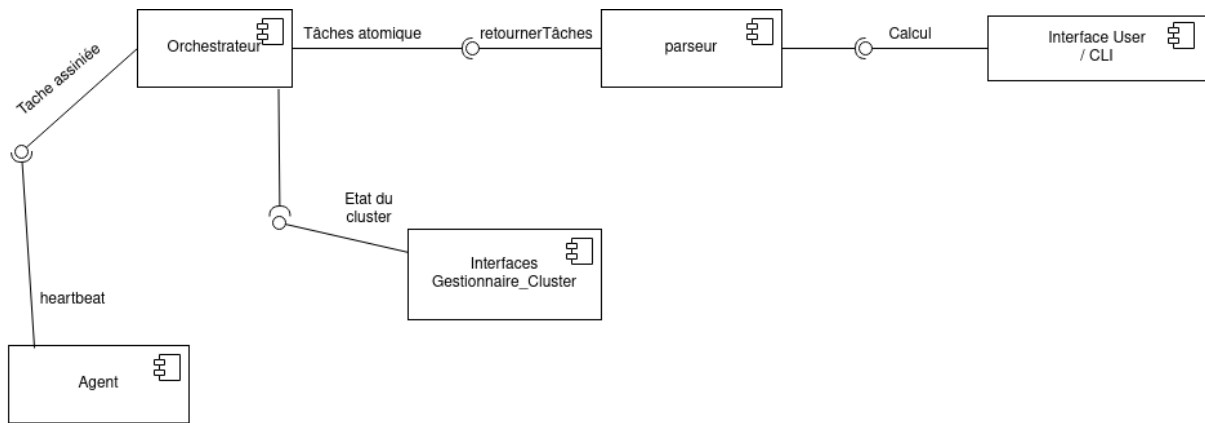


FIGURE 2.28 – Diagramme des composants

Recomposition des résultats

3. IMPLÉMENTATION ET RÉSULTATS

3.1 Conception et limites du modèle réseau classique (Network Agent Abstraction)

Dans le cadre d'un système multi-agents concurrent comme PARALLAX, l'implémentation de la communication réseau directe au sein de chaque thread applicatif pose des limites physiques et logicielles majeures :

- **Concurrence de binding** : Lorsque plusieurs threads tentent d'écouter ou d'initier des connexions sur la même interface réseau en utilisant des ports spécifiques, des conflits d'allocation d'adresses (erreur `EADDRINUSE`) surviennent inévitablement.
- **Mélange de flux d'entrée (Stream Mixing)** : Si plusieurs threads accèdent simultanément au même descripteur de socket sans synchronisation rigoureuse, les paquets TCP entrants sont entrelacés. Cela corrompt la sémantique des messages et rend leur désérialisation impossible.
- **Épuisement des ports** : La création et la destruction incessantes de sockets TCP par des threads à durée de vie courte sature la table des descripteurs système et la table de routage (sockets bloqués en état `TIME_WAIT`).

Pour surmonter ces contraintes, l'architecture de PARALLAX introduit une **Abstraction d'Agent Réseau** (`network_agent.c`). Chaque nœud physique exécute une unique instance d'Agent Réseau persistant qui centralise et orchestre toutes les communications physiques via trois threads spécialisés :

1. **socket_listener** (TCP) : Un thread d'écoute centralisé sur un port fixe (par exemple 9008 pour le Réceptionniste, 9005 pour le Master, 9006 pour le Worker, 9000 pour le Contrôleur). Il accepte les connexions TCP entrantes de manière asynchrone et les distribue.
2. **socket_sender** (TCP) : Un thread d'expédition unique qui consomme les messages réseau sortants et gère leur transmission physique sans interférence.
3. **udp_socket_listener** : Dédié à la réception des signaux UDP (notamment pour la découverte dynamique).

Le découplage entre les threads applicatifs et l'Agent Réseau est matérialisé par des **files de messages IPC System V** (implémentées dans `ms_queue.c`). Lorsqu'un thread applicatif souhaite envoyer un message réseau, il pousse un message typé dans la file IPC locale dédiée à l'Agent Réseau (**outgoing**). Inversement, l'Agent Réseau place les messages entrants dans les files IPC locales spécifiques aux types de messages applicatifs (ex. `PROG`, `HELLO`, `NODES`). Ce mécanisme asynchrone offre un tampon robuste qui élimine toute contention sur les ports réseau physiques.

3.2 L'Agent Réceptionniste

L'Agent Réceptionniste est la passerelle d'accès utilisateur au cluster. Ses responsabilités incluent l'exposition d'endpoints d'administration HTTP, la collecte des logs d'exécution transférés par le Contrôleur et la gestion de son cycle de vie autonome.

3.2.1 Architecture multi-thread de l'Agent Réceptionniste

L'Agent Réceptionniste repose sur l'exécution concurrente de 8 threads, décrits ci-dessous :

1. **Thread Principal** : Initialise le gestionnaire de files de messages IPC, démarre l'Agent Réseau sur le port 9008, crée la file locale `receptionist_out` et orchestre la découverte initiale.
2. **TCP Listener (Agent Réseau)** : Écoute sur le port 9008 pour recevoir les paquets TCP entrants (mises à jour d'IP, acquittements, logs) et les pousse dans la file IPC.
3. **TCP Sender (Agent Réseau)** : Dépille les messages de la file IPC locale et les envoie sur le réseau TCP.
4. **UDP Listener (Agent Réseau)** : Écoute sur le port 9009 pour recevoir les réponses UDP.
5. **HTTP Server** (`code_submission_listener_thread`) : Serveur HTTP léger écoutant sur le port 9010. Il expose les routes REST suivantes :
 - `/nodes` : Renvoie la liste et l'état des machines physiques du cluster.
 - `/node-logs/<uuid>` : Expose le journal système d'un nœud spécifique.
 - `/logs` : Renvoie l'historique global.
 - Soumission de code : Route POST acceptant le chargement de code source C.
6. **Query Master IP** (`receptionist_thread`) : Interroge le Contrôleur toutes les 5 secondes via un message `REQUEST_MASTER_IP_TYPE` pour obtenir l'IP du Master actif en cours, maintenant l'état interne de l'agent à jour.
7. **Log Receiver** (`log_receiver_thread`) : Lit les messages de type `PROG_LOG_TYPE` sur la file IPC System V et les écrit de manière persistante sur disque dans des fichiers sous le répertoire `logs/`.
8. **Heartbeat** (`receptionist_heartbeat_thread`) : Émet périodiquement toutes les 2 secondes un paquet de présence de rôle 4 (`ROLE_RECEPTIONIST`) à destination du Contrôleur.

La figure 3.1 modélise l'architecture de ces threads concourants.

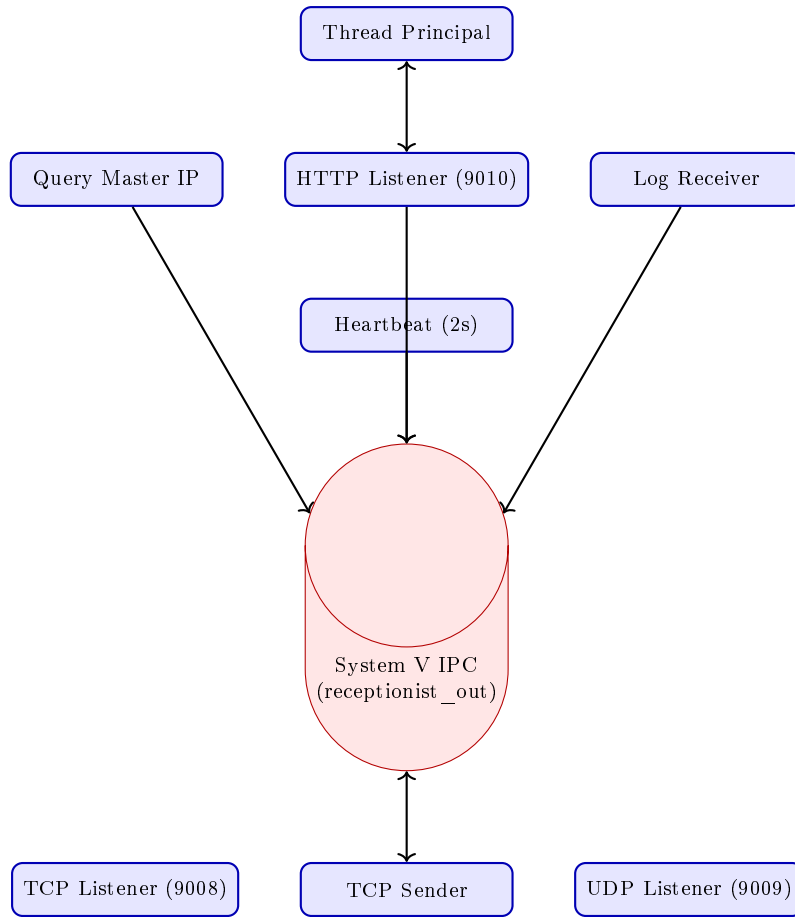


FIGURE 3.1 – Structure des threads de l'Agent Réceptionniste

3.2.2 Cycle de vie et découverte de l'Agent Réceptionniste

Lors du démarrage, l'agent génère un identifiant unique (UUID) persistant en lisant la source d'entropie système `/dev/urandom`, puis stocke cette valeur localement dans le fichier `.receptionist_uuid`.

L'agent entre ensuite dans une phase de découverte UDP active (`discover_controller`) : il envoie un message broadcast `MSG_HELLO` contenant son UUID et son port réseau local (port 9008) sur le port UDP 9001. Le Contrôleur intercepte ce message, enregistre le Réceptionniste et lui répond directement par une trame UDP unicast de type `HELLO_TYPE` contenant la chaîne `IP:<controller_ip>` vers le port 9009. Dès réception, l'agent enregistre l'IP du Contrôleur et démarre ses threads fonctionnels.

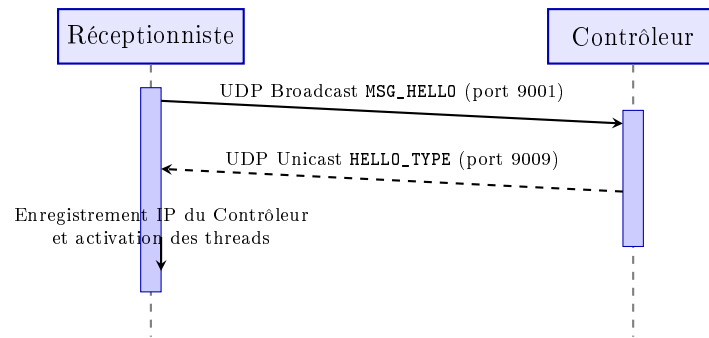


FIGURE 3.2 – Diagramme de séquence : Protocole de découverte de l'Agent Réceptionniste

3.3 L'Agent Master

L'Agent Master est le coordinateur des calculs. Il centralise les soumissions de code, découpe les tâches, interroge le contrôleur pour localiser les ressources, distribue les sous-calculs et fusionne les résultats.

3.3.1 Architecture multi-thread de l'Agent Master

L'Agent Master structure son exécution autour d'un ensemble de threads concurrents :

- **Thread Principal** : Lance l'Agent Réseau sur le port 9005 et initialise le gestionnaire d'exécutions.
- **Threads de l'Agent Réseau** (`socket_listener`, `socket_sender`, `udp_socket_listener`).
- **Prog Listener** (`master_thread_start`) : Écoute les nouveaux programmes sur la file de messages IPC locale `PROG`. À chaque réception, il instancie un thread d'orchestration dédié (`prog_listener_func`) pour compiler et exécuter le code de façon non bloquante.
- **Pool Parallax Team** : Pool dynamique de threads créés par l'orchestrateur. Chaque thread du pool pilote et surveille un Worker distant spécifique pour une tâche donnée.

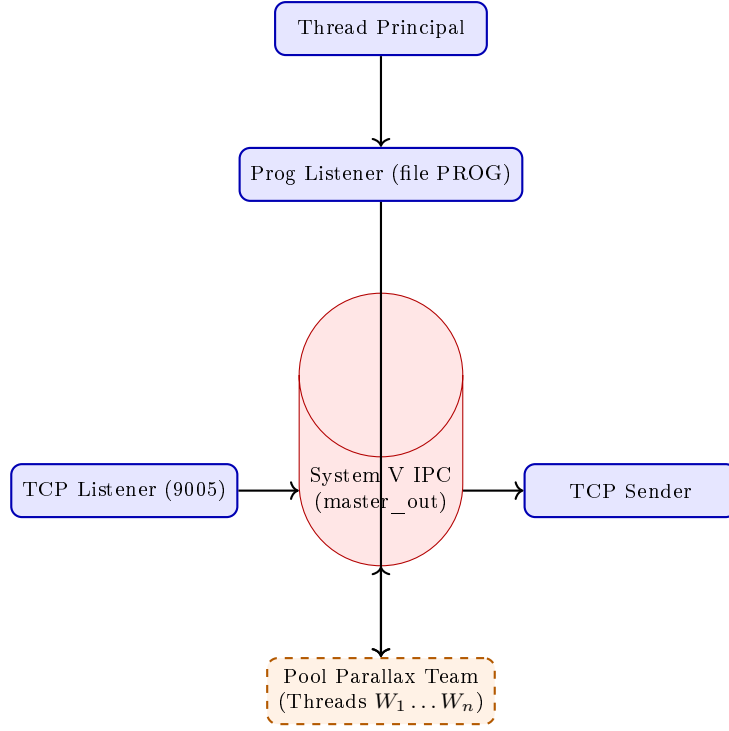


FIGURE 3.3 – Structure des threads de l’Agent Master

3.3.2 Workflow complet d’Orchestration et d’Exécution

Lorsqu’un programme est reçu, le Master exécute les phases suivantes de manière séquentielle (voir Figure 3.6) :

1. **Parsing du code source** : Le code source C est analysé par le parseur C++ de PARALLAX (Parser/build/mytool). Celui-ci génère un code intermédiaire annoté (`_parsed.c`) où les directives de parallélisme sont traduites en structures C standard.
2. **Interrogation du Contrôleur** : Le Master émet une requête réseau de type NODES sur le port 9000 du Contrôleur et suspend son exécution sur une file IPC locale temporaire (`nodes_mq_name`). Le Contrôleur renvoie un tableau de structures `MachineMetrics` de tous les Workers actifs.
3. **Tri par Score de Performance** : Le Master trie les Workers disponibles par ordre décroissant de score de performance. La fonction de score d’un nœud n_i est calculée par le Contrôleur et partagée via sa télémétrie :

$$\text{score}(n_i) = \alpha \cdot \text{norm_cpu}_i + \beta \cdot \text{norm_ram}_i + \gamma \cdot \text{norm_net}_i + \delta \cdot c_queue_i \quad (3.1)$$

Où les coefficients de pondération réels sont définis par $\alpha = 0.5$, $\beta = 0.3$, $\gamma = 0.1$, $\delta = 0.1$, et les composantes de capacité sont :

- Capacité de calcul CPU : $C_{\text{cpu}} = \text{cores} \times \text{threads} \times \text{freq_mhz} \times (1 - \text{cpu_usage}/100) \times \frac{1}{1 + \text{load1m}}$
- Capacité RAM disponible : $C_{\text{ram}} = \text{mem_available_mb}$
- Capacité réseau : $C_{\text{net}} = \text{network_bandwidth_mbps}$

— Capacité de file d'attente locale : $c_{\text{queue}} = \frac{1}{1 + \text{queue_len}}$

Les valeurs norm_X_i représentent la normalisation de la capacité par rapport au maximum observé dans le cluster : $\text{norm_X}_i = \frac{C_X(i)}{\max_j C_X(j)}$. Si un nœud est marqué comme surchargé (`is_overloaded`), son score est forcé à 0.0.

4. **Distribution des sous-calculs (SCATTER)** : Le Master segmente le paramètre d'entrée marqué `PARALLAX_SCATTER` proportionnellement aux poids de performance calculés des nœuds sélectionnés. Il génère les structures d'assignation `task_assignment`.
5. **Initialisation de la Parallax Team** : Le Master alloue une structure `team` contenant une barrière de synchronisation basée sur les verrous de bas niveau Linux (`futex`) et lance un thread par Worker.
6. **Handshake de Communication à 3 étapes** : Chaque thread de la team gère la liaison avec son Worker via le protocole suivant :
 - **CHCK** : Envoi d'une requête demandant si le binaire du programme est déjà compilé sur le Worker.
 - **PROG** : Si le Worker répond `NONE`, le Master lui envoie le code source complet à compiler.
 - **TASK** : Envoi de la tâche contenant le nom de la fonction à exécuter et le bloc de paramètres d'entrée sérialisés. Le thread bloque en attente de la réponse réseau.
7. **Barrière de Synchronisation** : Après avoir reçu son résultat partiel, chaque thread se bloque sur `barrier_wait` (synchronisation par `futex`).
8. **Réduction et Agrégation** : Une fois tous les threads libérés, le thread principal applique la fonction de réduction dynamique (par exemple `sum_reduce`) sur l'accumulateur des résultats partiels pour générer le résultat consolidé.
9. **Shipment des logs** : Le Master récupère le fichier de log local, l'encapsule dans un paquet de type `PROG_LOG_TYPE` et l'envoie au Contrôleur pour archivage.

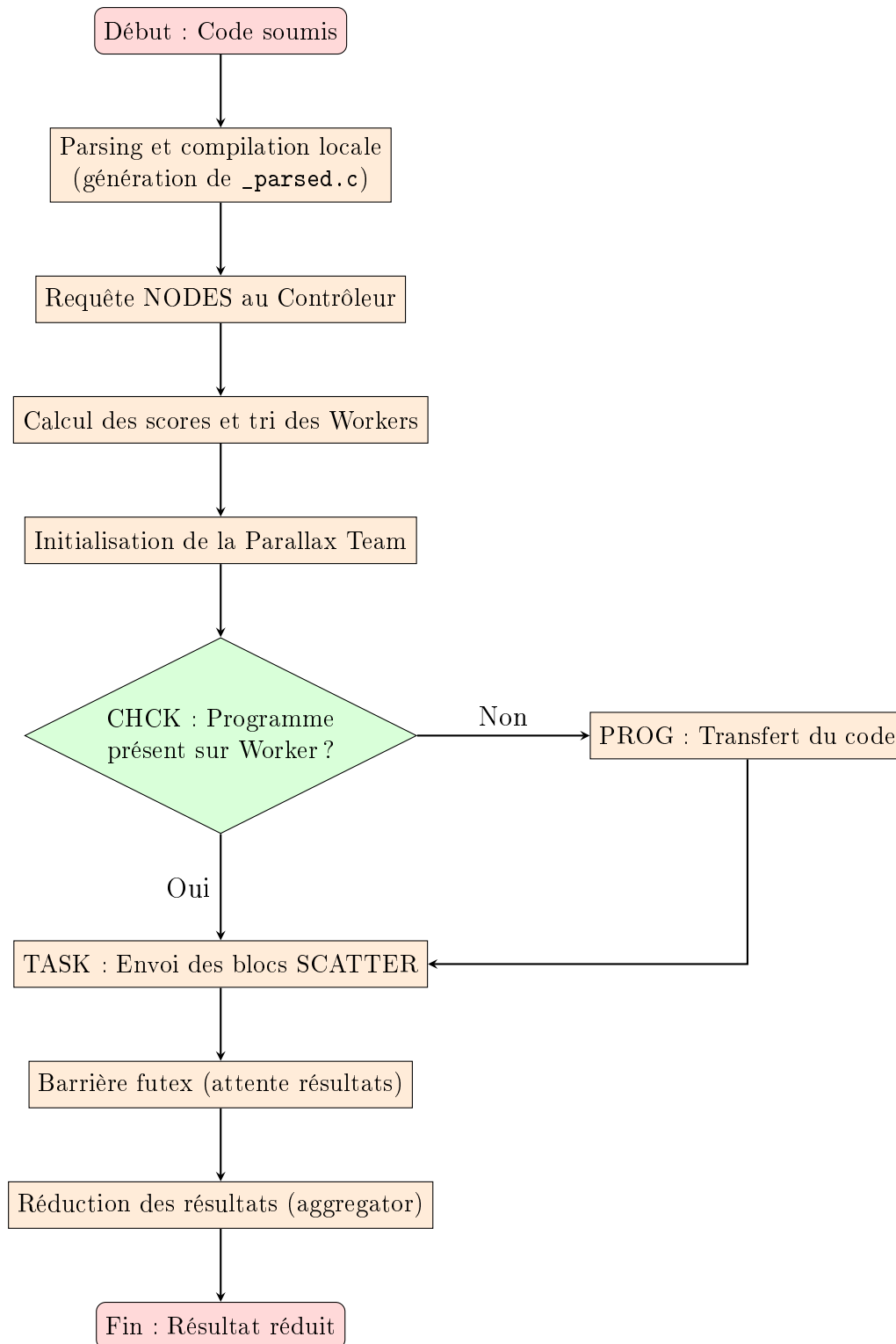


FIGURE 3.4 – Diagramme d'activité : Workflow d'exécution de l'Agent Master

3.4 L'Agent Worker

L'Agent Worker s'exécute sur chaque nœud de calcul du cluster. Il est responsable de la compilation dynamique des tâches, de leur isolation à l'exécution et du contrôle local des

ressources.

3.4.1 Architecture multi-thread de l'Agent Worker

L'Agent Worker s'appuie sur les threads concurrents suivants :

- **Thread Principal** : Lance l'Agent Réseau sur le port 9006, crée les files IPC locales et attend les ordres du Master.
- **Agent Réseau** (`socket_listener`, `socket_sender`, `udp_socket_listener`).
- **Execution Thread** : Traite le protocole CHCK/PROG/TASK de manière séquentielle pour un Master donné.
- **Log Monitor Thread** : Surveille la taille du fichier journal généré par les processus exécutés.

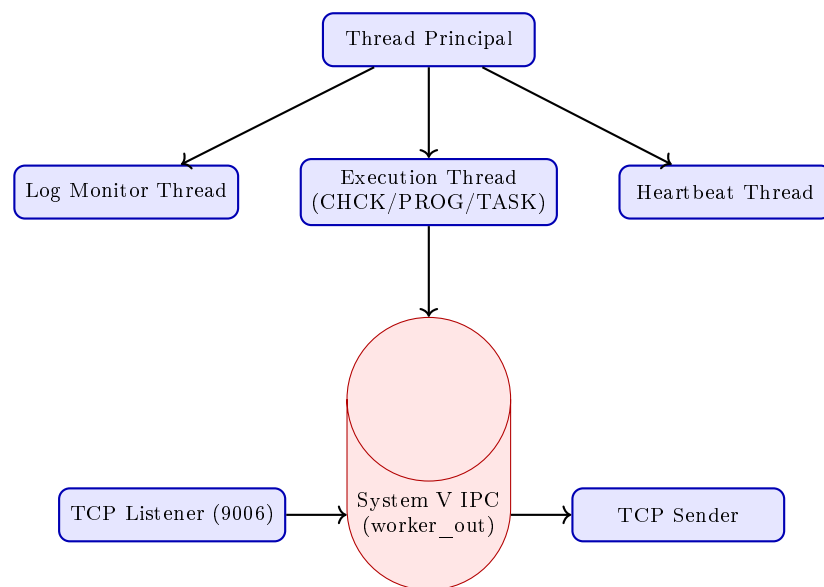


FIGURE 3.5 – Structure des threads de l'Agent Worker

3.4.2 Compilation Dynamique, Isolation et Rotation des Logs

Compilation dynamique sans `main()` Afin d'exécuter des fonctions spécifiques reçues dynamiquement sans redémarrer le daemon Worker, le code compilé n'intègre pas la fonction traditionnelle `main()`. Le Worker compile les programmes reçus sous forme de binaires autonomes en injectant le drapeau de liaison GCC `-Wl,-e,worker_entry`. Ce drapeau redéfinit le point d'entrée d'exécution au niveau de la fonction `worker_entry()`, qui prend en charge la désérialisation locale des paramètres.

Isolation de l'exécution Pour empêcher qu'un calcul utilisateur corrompu ne fasse planter le daemon principal du Worker, chaque exécution de tâche est isolée dans un processus séparé :

1. Le Worker effectue un appel système `fork()`.
2. Le processus fils redirige son flux de sortie standard (`stdout/stderr`) vers un fichier de log via des descripteurs de fichiers dupliqués (`dup2()`).

3. Le fils charge et exécute le binaire compilé de la tâche via un appel système `execv()`.
4. Le processus père (le daemon Worker) surveille l'exécution du fils et récupère le code de retour à la fin.

Rotation et surveillance des logs Le thread `Log Monitor` vérifie en tâche de fond la taille du fichier de log associé à l'exécution en cours. Si la taille du fichier dépasse un seuil de **512 Ko**, le thread tronque le fichier et effectue une rotation pour préserver l'espace disque du nœud de calcul et éviter tout déni de service local.

3.5 L'Agent Contrôleur

L'Agent Contrôleur est le superviseur du cluster. Il centralise les battements de cœur, maintient la table d'état globale des nœuds et détecte les pannes.

3.5.1 Architecture multi-thread de l'Agent Contrôleur

Pour traiter les messages asynchrones du cluster sans blocage, le Contrôleur maintient une structure hautement parallèle comprenant **15 threads simultanés** :

- **Thread Principal** : Démarre l'Agent Réseau sur le port 9000.
- **Agent Réseau** (`socket_listener`, `socket_sender`, `udp_socket_listener`).
- **11 Threads de Traitement (Handlers IPC)** :
 - `hello_func` : Traite les requêtes HELLO UDP, enregistre les nœuds et renvoie l'IP du Contrôleur.
 - `heartbeat_receiver_func` : Reçoit les pulsations UDP et met à jour l'horodatage de présence.
 - `statecapture_init_func` : Enregistre les métriques matérielles initiales lors de la première connexion.
 - `statecapture_func` : Met à jour les métriques dynamiques des nœuds (CPU, RAM, bande passante).
 - `node_log_receiver_func` : Reçoit les logs système des Workers et les écrit dans `node_logs/`.
 - `get_node_log_func` : Transmet les logs archivés à la demande de l'Agent Réceptionniste.
 - `log_relay_func` : Redirige les journaux d'exécution reçus des Masters vers l'Agent Réceptionniste connecté.
 - `request_master_ip_func` : Répond aux demandes de l'IP du Master formulées par le Réceptionniste.
 - Trois threads de maintenance interne et de surveillance périodique.

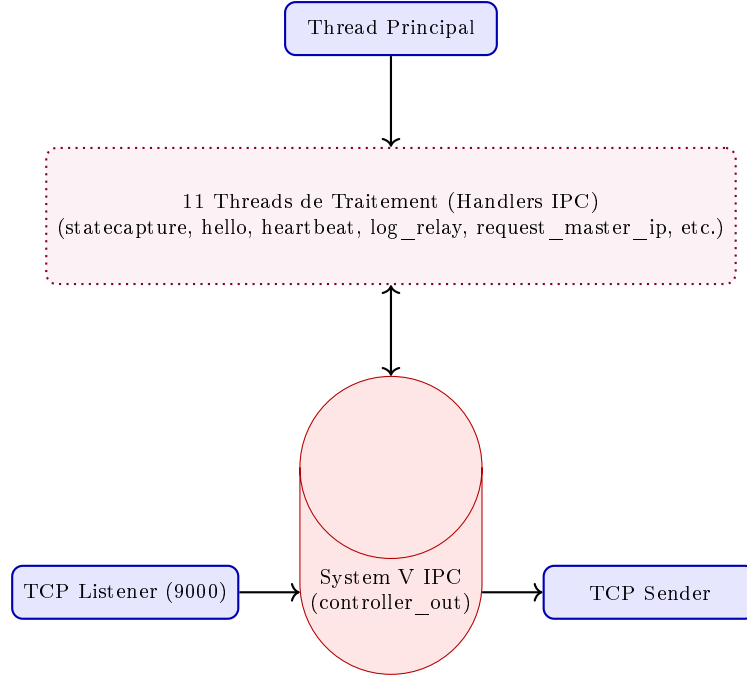


FIGURE 3.6 – Structure des threads de l'Agent Contrôleur

3.5.2 Mécanisme de Watchdog et Détection de pannes

Le Contrôleur exécute périodiquement un algorithme de vérification d'état (Watchdog) sur sa table des nœuds connectés (`g_node_table`). Pour chaque nœud n_j , la durée écoulée depuis le dernier heartbeat reçu est calculée : $\Delta_j = t - \text{last_heartbeat}_j$.

Les transitions d'état sont appliquées strictement selon les règles temporelles définies en configuration :

1. **Transition en état SUSPECT** : Si $\Delta_j \geq T_{\text{suspect}} = 4$ secondes, le Contrôleur marque le nœud comme `NODE_SUSPECT` et initie une requête de confirmation Gossip vers les autres nœuds.
2. **Transition en état EN PANNE** : Si $\Delta_j \geq T_{\text{failed}} = 8$ secondes, le Contrôleur confirme la défaillance, bascule l'état du nœud à `NODE_EN_PANNE` et retire le nœud du pool des Workers actifs. Si le nœud en panne était le Master actif du cluster, le Contrôleur initie immédiatement une élection pour désigner un nouveau Master parmi les Workers fonctionnels restants.

3.6 Trace complète d'un cycle de calcul

Le fonctionnement de PARALLAX s'articule autour des 9 étapes suivantes lors d'un calcul utilisateur :

1. **Soumission HTTP** : L'utilisateur soumet son code C annoté (contenant une directive de parallélisme comme un bloc d'exécution parallèle) à l'Agent Réceptionniste via une requête HTTP POST sur le port 9010.
2. **Transfert au Master** : Le Réceptionniste interroge le Contrôleur pour obtenir l'IP du Master actif, puis pousse le programme reçu dans la file IPC réseau du Master.

3. **Parsing & Compilation** : Le Master reçoit le programme via son thread `master_thread_start`, le parse en C++ pour générer le code source distribué, et le compile sous forme de binaire de contrôle.
4. **Récupération des ressources** : Le Master envoie une requête `NODES` au Contrôleur et récupère les métriques en temps réel des Workers actifs du cluster.
5. **Calcul des scores & Planification** : Le Master trie les Workers par leur score de performance calculé, décide de la distribution des données du bloc `SCATTER` et crée la Parallax Team d'exécution.
6. **Handshake CHCK-PROG-TASK** : Chaque thread de la team gère la liaison avec son Worker via le protocole suivant :
 - **CHCK** : Envoi d'une requête demandant si le binaire du programme est déjà compilé sur le Worker.
 - **PROG** : Si le Worker répond `NONE`, le Master lui envoie le code source complet à compiler.
 - **TASK** : Envoi de la tâche contenant le nom de la fonction à exécuter et le bloc de paramètres d'entrée sérialisés. Le thread bloque en attente de la réponse réseau.
7. **Barrière de Synchronisation** : Après avoir reçu son résultat partiel, chaque thread se bloque sur `barrier_wait` (synchronisation par futex).
8. **Réduction et Agrégation** : Une fois tous les threads libérés, le thread principal applique la fonction de réduction dynamique (par exemple `sum_reduce`) sur l'accumulateur des résultats partiels pour générer le résultat consolidé.
9. **Shipment des logs** : Le Master récupère le fichier de log local, l'encapsule dans un paquet de type `PROG_LOG_TYPE` et l'envoie au Contrôleur pour archivage.

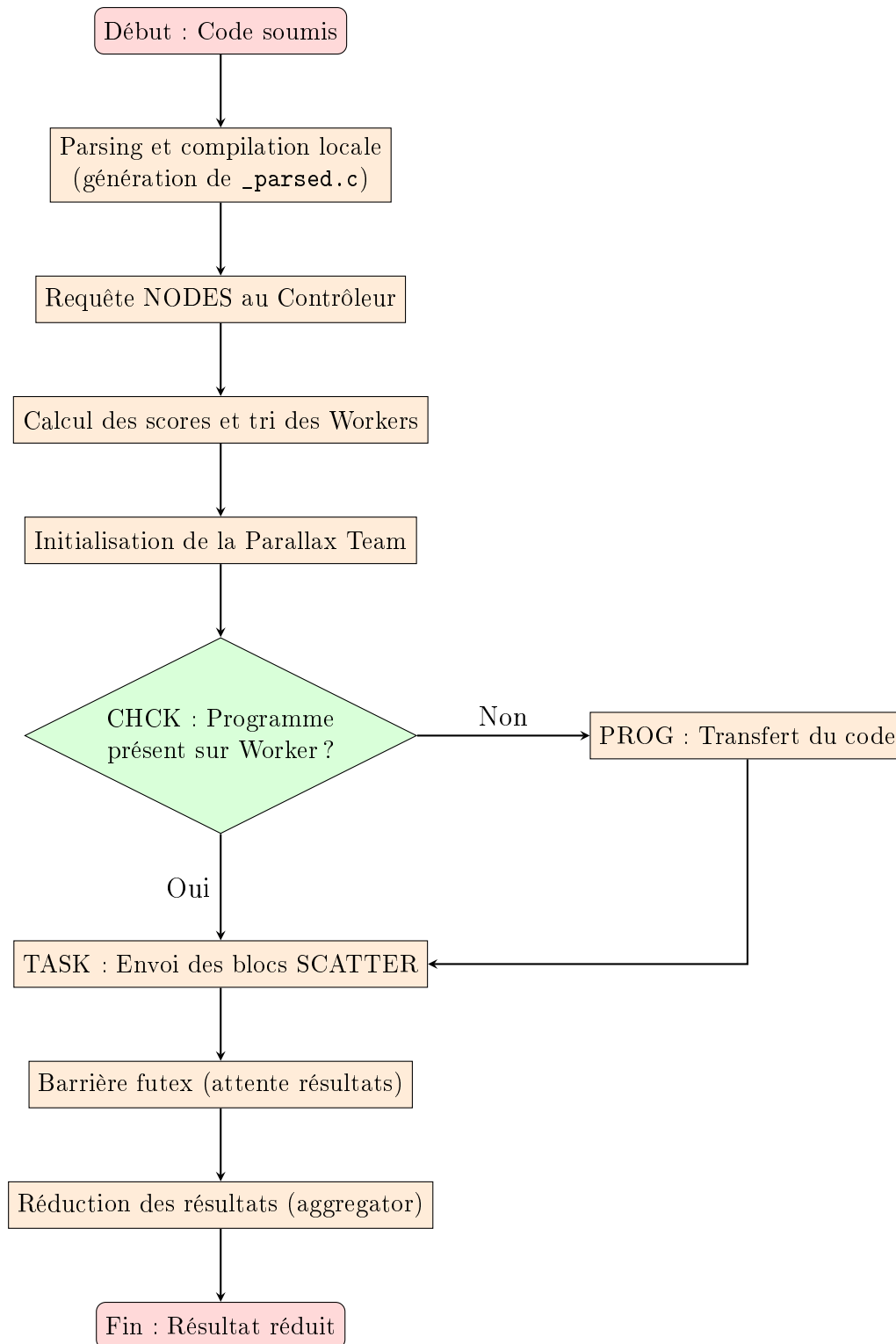


FIGURE 3.7 – Diagramme d'activité : Workflow d'exécution de l'Agent Master

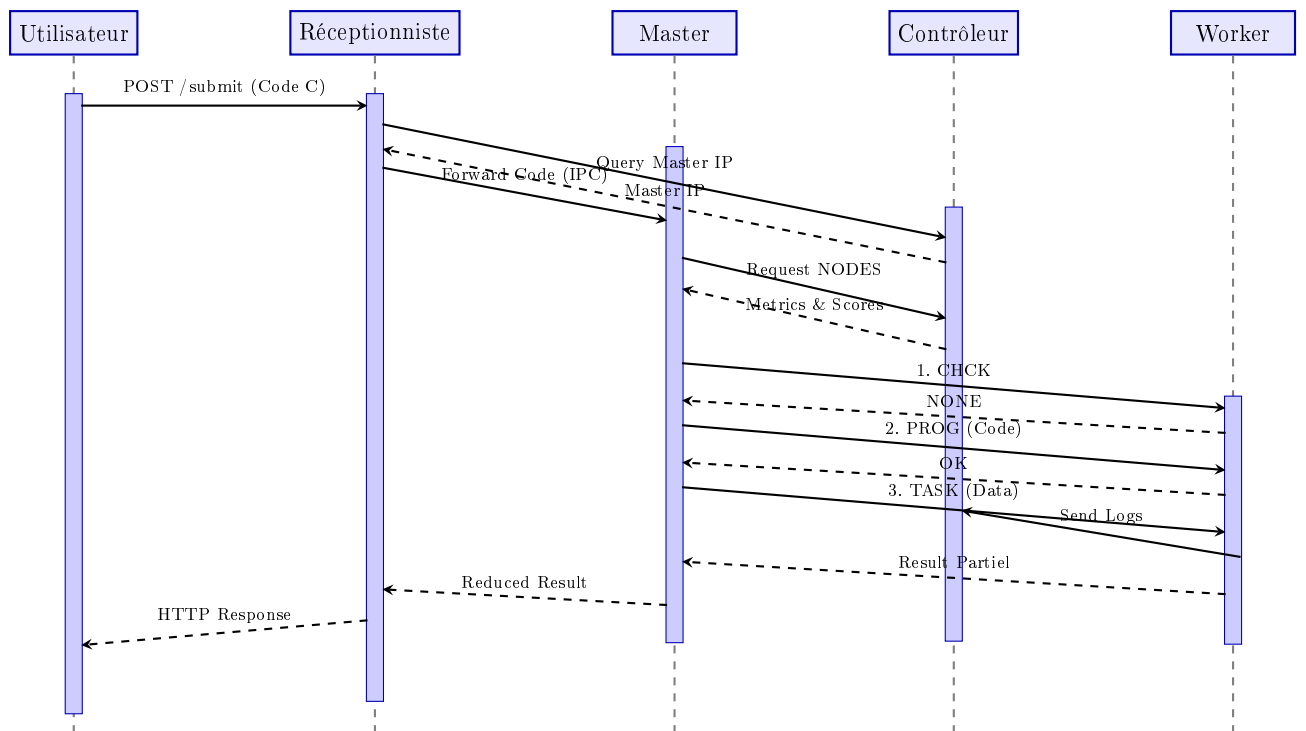


FIGURE 3.8 – Diagramme de séquence : Interaction globale lors d'un cycle de calcul

CONCLUSION GÉNÉRALE

Le projet PARALLAX, réalisé dans le contexte académique de l'École Nationale Supérieure Polytechnique de Yaoundé, a permis de concevoir et de modéliser une infrastructure de calcul distribué basée sur l'exploitation de machines locales hétérogènes et sous-utilisées.

L'approche adoptée, fondée sur les Systèmes Multi-Agents, a facilité la structuration du système en entités autonomes assurant les fonctions de coordination, d'exécution et de supervision. Cette modélisation a contribué à améliorer la résilience du système face aux défaillances et à mieux organiser la répartition des charges dans un environnement distribué.

Les résultats obtenus montrent que l'architecture mise en place permet d'améliorer l'utilisation des ressources disponibles et de réduire les temps d'exécution pour certaines classes de tâches. Le système valide également sa capacité à maintenir son fonctionnement en présence de pannes partielles.

Les perspectives de ce travail incluent l'amélioration des politiques d'ordonnancement, notamment par l'intégration de techniques d'apprentissage automatique afin d'optimiser dynamiquement l'allocation des tâches au sein du cluster.

Références

BIBLIOGRAPHIE

- [1] D. P. Anderson, “SETI@home : An Experiment in Public-Resource Computing,” *Communications of the ACM*, vol. 45, no. 11, pp. 56–61, 2002.
- [2] D. Thain, T. Tannenbaum, and M. Livny, “Distributed Computing in Practice : The Condor Experience,” *Concurrency and Computation : Practice and Experience*, vol. 17, no. 2–4, pp. 323–356, 2005.
- [3] A. S. Tanenbaum and M. Van Steen, *Distributed Systems : Principles and Paradigms*, 2nd ed. Prentice Hall, 2007.
- [4] M. Wooldridge, *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2009.
- [5] H. Topcuoglu, S. Hariri, and M.-Y. Wu, “Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, 2002.
- [6] B. Burns, J. Beda, and K. Hightower, *Kubernetes : Up and Running*. O’Reilly Media, 2019.
- [7] H. Garcia-Molina, “Elections in a Distributed Computing System,” *IEEE Trans. Computers*, vol. C-31, no. 1, pp. 48–59, 1982.
- [8] Cours d’Agents Intelligents et Systèmes Multi-Agents, *Chapitre 1 : Agents et Systèmes Multi-Agents — Définitions et Caractéristiques*, A. M. Florea, Université Politehnica de Bucarest / ENSPY, 2024.
- [9] Cours d’Agents Intelligents et Systèmes Multi-Agents, *Chapitre 2 : Typologie des Agents*, ENSPY — Génie Informatique, 2024.

Annexes